

The Secrets Must Not Flow: Scaling Security Verification to Large Codebases (extended version)

Anonymous Author(s)

Abstract—Existing program verifiers can prove advanced properties about security protocol implementations, but are difficult to scale to large codebases because of the manual effort required. We develop a novel methodology called **DIONON** that addresses this challenge by splitting the codebase into the protocol implementation (the **CORE**) and the remainder (the **APPLICATION**). This split allows us to apply powerful semi-automated verification techniques to the security-critical **CORE**, while fully-automatic static analyses scale the verification to the entire codebase by ensuring that the **APPLICATION** cannot invalidate the security properties proved for the **CORE**. The static analyses achieve that by proving *I/O independence*, i.e., that the I/O operations within the **APPLICATION** are independent of the **CORE**’s security-relevant data (such as keys), and that the **APPLICATION** meets the **CORE**’s requirements. We have proved **DIONON** sound by first showing that we can safely allow the **APPLICATION** to perform I/O independent of the security protocol, and second that manual verification and static analyses soundly compose. We evaluate **DIONON** on two case studies: an implementation of the signed Diffie-Hellman key exchange and a large (100k+ LoC) production Go codebase implementing a key exchange protocol for which we obtained secrecy and injective agreement guarantees by verifying a **CORE** of about 1 % of the code with the auto-active program verifier Gobra in less than three person months.

1. Introduction

Security protocols such as TLS or Signal ensure security and privacy for browsing the web, sending private messages, and using cloud services. It is, thus, crucial that these ubiquitous and critical protocols are designed *and* implemented correctly.

Automatic protocol verifier tools such as Tamarin [1], [2] and ProVerif [3] make it viable to formally verify protocol *models*. Their applications to TLS [4], EMV [5], Signal [6], and 5G [7], [8] demonstrate that they can handle realistic protocols. However, proving protocol *models* secure does not result in secure *implementations* on its own. Coding errors such as missing bounds checks (e.g., causing the Heartbleed [9] bug), omitted protocol steps (as in the Matrix SDK [10]), or ignored errors (e.g., returned by a TLS library [11], [12]) may invalidate all security properties proven for the corresponding models.

Verifying security properties for protocol *implementations* is possible as well [13], [14], [15]. For instance, Ar-

quint et al. [14] first verify security properties for a Tamarin model of the protocol in the presence of a Dolev-Yao (DY) attacker [16] fully controlling the network. Then, they prove that the protocol implementation refines this model, i.e., that the model justifies every I/O operation performed by the implementation. Refinement guarantees that the implementation inherits the security properties proven for the model.

Existing approaches to verifying protocol implementations are sound *only* if they are applied to the *entire* implementation. Verifying only a subset of the codebase is unsound, and would fail to prevent, e.g., code seemingly unrelated to a security protocol accidentally logging key material [17], [18]. However, the required expertise and annotation overhead make it infeasible to verify entire *production* codebases, which often consist of hundreds of thousands of lines of code.

This work. We present **DIONON**¹, a proved-sound methodology that scales verification of security properties to large production codebases. **DIONON** works with codebases where a small, syntactically-isolated component implements a security protocol, whose security argument can be made separately from the rest of the code. Our methodology decomposes the overall codebase into this protocol implementation (the **CORE**) and the remainder (the **APPLICATION**).

This decomposition allows us to apply different verification techniques to the two parts. We verify the **CORE** using Arquint et al.’s approach to show refinement w.r.t. a verified Tamarin model, which requires precise reasoning about, e.g., the payloads of I/O operations. Instead of applying the same annotation-heavy approach to the **APPLICATION**, we use automatic static analyses to ensure that security-relevant data of the **CORE** (in particular, secrets such as keys) does not influence any I/O operation within the **APPLICATION**. If this *I/O independence* holds, the **APPLICATION** cannot perform any I/O operations that could interfere with the protocol and invalidate its proven security. Additionally, we use static analyses to prove that the **APPLICATION** satisfies the assumptions made for the proof of the **CORE**, in particular, that the preconditions of **CORE** functions hold when called from the **APPLICATION** and that the **APPLICATION** does not violate any invariants of **CORE** data structures. These checks ensure that the proofs of the **CORE** and the **APPLICATION** compose soundly. Consequently, the entire codebase refines

1. Diodon is a genus of fish known for their inflation capabilities. Erecting spines and scaling their volume by a multiple provide security, like our verification methodology.

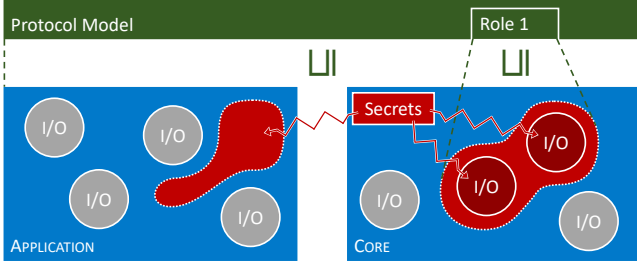


Figure 1. The DIODON methodology. We partition the codebase (blue) into the module implementing a protocol (CORE) and the remaining codebase (APPLICATION). We prove that the CORE refines a particular role of the verified protocol model (green) by auto-active verification. We apply static analyses to the entire codebase to enforce that secrets (red) do not influence (red arrows) the I/O operations (gray circles) of the APPLICATION and to ensure that the APPLICATION cannot invalidate the security properties proved for the CORE. Consequently, the entire codebase refines the protocol model and, thus, enjoys all security properties proved for that model.

the protocol model and enjoys all security properties proved for the model. DIODON significantly reduces the proof effort of verifying software that contains protocol implementations. Fig. 1 illustrates our approach.

We prove I/O independence for the APPLICATION by executing an automatic taint analysis on the entire codebase to identify I/O operations that are possibly affected by secrets and checking that all such operations are within the CORE, which shows that the codebase’s decomposition is valid and the CORE is sufficiently large. Note that it would be too restrictive to enforce that all secrets are confined within the CORE. In most implementations, secrets exist outside of the CORE, e.g., the APPLICATION might have access to secrets either via program inputs or the CORE’s state (red area within the APPLICATION in Fig. 1). It is therefore essential to ensure (via a whole-program analysis) that the APPLICATION does not *use* these secrets to violate the security properties of the Tamarin model.

Most I/O operations within the CORE correspond to a protocol step and are relevant for proving refinement w.r.t. a protocol model. In production code, however, the CORE might also contain operations irrelevant to the protocol, such as logging a protocol step. To reduce the verification effort further, we also check I/O independence *within* the CORE to classify each I/O operation based on whether it depends on secrets occurring in a protocol run (dark red circles in Fig. 1) or not (gray circles). The former need to be considered during the refinement proof, while the latter can safely be ignored. Besides simplifying the refinement proof, this classification allows the abstract protocol model to remain concise.

We prove refinement of the CORE w.r.t. a protocol model using an *auto-active* program verifier [19]. These tools take as input an implementation annotated with specifications such as pre- and postconditions and loop invariants, and attempt to verify the implementation automatically using an SMT solver.

Auto-active verification is generally sound only if it is applied to the entire codebase because *all* callers of a

function must establish its precondition and *all* functions must preserve data structure invariants. To ensure that our approach is sound while avoiding this requirement for the APPLICATION, we design our methodology such that static analyses automatically discharge the proof obligations on the APPLICATION. Nevertheless, our methodology is flexible enough to permit complex interactions between the CORE and APPLICATION, e.g., through concurrency and callbacks. Some assumptions remain, in particular, the absence of data races and undefined behavior; we discuss those in Sec. 4.4.

We prove DIODON sound, providing a blueprint for combining the distinct formalisms of auto-active verifiers and static analyses. First, we prove that a DY attacker can simulate all *secret-independent* I/O operations. Consequently, if a Tamarin model permits every *secret-dependent* I/O operation in a codebase, then this codebase refines the model. Second, we show that DIODON allows reasoning about these *secret-dependent* I/O operations *without* verifying the entire codebase. I.e., we construct the corresponding proof for the entire codebase by starting from the proof for the CORE, which we obtain from auto-active verification, and discharging the remaining proof obligations using our static analyses.

We evaluate DIODON on two Go implementations, the signed Diffie-Hellman (DH) key exchange and the Amazon Web Services (AWS) Systems Manager Agent [20], a large (100k+ LoC) codebase widely used by AWS customers. Part of the latter codebase implements a protocol for encrypted shell sessions. We prove secrecy for and injective agreement on the session keys established by both protocols. For the AWS codebase, DIODON allowed us to limit auto-active verification to only about 1% of the entire codebase, which took less than three person months. This demonstrates that DIODON enables, for the first time, the verification of strong security properties at the scale of production codebases.

Contributions. We make the following contributions:

- We present a scalable verification methodology for implementations of security protocols within large codebases, which applies to any codebase with a clear distinction between the protocol core and the rest of the code.
- We identify I/O independence, enabling concise protocol models for complex implementations.
- We show how to use static analyses to automatically discharge the CORE’s proof obligations, enabling DIODON to scale to large codebases.
- We prove the soundness of I/O independence w.r.t. a DY attacker, and the soundness of DIODON’s combination of auto-active verification and static analyses.
- We evaluate our methodology on two case studies, an implementation of the signed DH key exchange and the AWS Systems Manager Agent, to demonstrate that DIODON scales to large, production codebases.

2. Running Example of DIODON

We demonstrate the core ideas of DIODON on a sample program in the Go programming language, which implements a simple message authentication code (MAC) protocol

```

1 package core
3 type Chan struct {
4     psk []byte
5     cb Cb
6 }
8 type Cb = func(msg []byte)
10 //@ req acc(msg, 1)
11 //@ func CbSpec(msg []byte)

13 //@ req cb != nil ==> cb implements CbSpec{}
14 //@ pres psk != nil ==> acc(psk, 1)
15 //@ ens Inv(c)
16 func InitChannel(psk []byte, cb Cb) (c *Chan) {
17     //@ inhale AliceIOPermissions()
18     c = &Chan{append([]byte(nil), psk...), cb}
19     go continuousRecv(c)
20     return c
21 }

23 //@ pres c != nil ==> Inv(c)
24 //@ pres msg != nil ==> acc(msg, 1)
25 func Send(c *Chan, msg []byte) {
26     if c == nil || msg == nil { return }
27     fmt.Printf("Send_%x\n", msg)
28     packet := append(msg, HMAC(msg, c.psk)...)
29     sendToNetwork(packet)
30 }

32 /*@ pred Inv(c *Chan) {
33     c != nil && acc(c, 1/2) &&
34     acc(c.psk, 1/2) && AliceIOPermissions() &&
35     (c.cb != nil ==> c.cb implements CbSpec{})
36 } @*/

```

Figure 2. Sample CORE for a simple MAC communication. We omit the continuousRecv goroutine’s implementation that invokes the c.cb closure (if non-`nil`) whenever a message has been received. We simplify the representation of I/O permissions, which describe permitted protocol-relevant I/O operations, and omit proof-related statements.

that sends and receives signed messages using a pre-shared key. First, we split the codebase into CORE and APPLICATION following function boundaries. We make the CORE as small as possible to reduce auto-active verification efforts while making sure that the entire protocol implementation is contained therein and that we can define an invariant for the CORE’s API with which the APPLICATION interacts.

We model the protocol and prove security properties with the Tamarin protocol verifier [1], [2]. The goal is to prove that the entire program, i.e., the composition of the CORE and APPLICATION, refines the Tamarin model and, thus, satisfies the same security properties as the protocol model. We auto-actively verify the CORE using Gobra [21] and apply the automatic Argot [22] static analyses to the entire codebase.

CORE. The CORE (Fig. 2) consists of a struct definition, two API functions, `InitChannel` and `Send`, which access this struct, and a predicate `Inv` that represents the separation logic [23] invariant used to verify the functions. The definition of `Inv` includes permissions to access the struct fields and the pre-shared key’s bytes. Separation logic controls heap access with these permissions to reason about side effects and to prove data-race freedom, as detailed

```

1 package main
3 import . "core"
5 func main(psk []byte) {
6     cb := func(m []byte) {fmt.Printf("%x\n", m)}
7     c := InitChannel(psk, cb)
8     Send(c, []byte("hello_world"))
9     fmt.Printf("Log:_message_sent.\n")
10    // fmt.Printf("%v\n", c)
11 }

```

Figure 3. Sample APPLICATION that is a client of Fig. 2. We omit parsing of command line arguments for presentation purposes and, thus, assume that `psk` stores the parsed pre-shared key.

```

1 rule Alice_Send:
2     let packet = <msg, sign(msg, psk)> in
3     [ Alice_1(rid, A, B, psk), In(msg) ]
4     --->
5     [ Alice_1(rid, A, B, psk), Out(packet) ]
6 rule Alice_Recv:
7     let packet = <msg, sign(msg, psk)> in
8     [ Alice_1(rid, A, B, psk), In(packet) ]
9     --->
10    [ Alice_1(rid, A, B, psk), Out(msg) ]

```

Figure 4. Tamarin model excerpt for the MAC protocol implemented in Fig. 2.

in Sec. 3.2. Accessibility predicates (`acc`) represent permissions in specifications. Their first argument indicates the heap location and the second argument characterizes the permitted access: a value of 1 provides exclusive read and write access, and any value strictly between 0 and 1 provides read-only access that might be shared. For instance, `acc(msg, 1)` on line 24 passes the permission to write the contents of `msg` (if it is non-`nil`) from a caller to function `Send`, and back to the caller when the function returns. Pre- and postconditions start with the keyword `req` and `ens`, respectively, and we use `pres` as syntactic sugar for properties that are preserved, that is, act as pre- and postconditions.

To receive incoming packets, the CORE spawns a goroutine (lightweight thread) on line 19 executing the function `continuousRecv`. We omit its implementation in the figure for space reasons. The goroutine repeatedly calls a blocking receive operation, checks the MAC’s validity, and on success calls the closure that is stored in the struct field `cb` as a callback. If the callback is non-`nil`, it delivers the resulting message to the APPLICATION.

We verify the CORE for any callback closure that satisfies the specification `CbSpec` (cf. line 13 & 10–11), which states that a caller must pass permission for modifying the message to the closure when invoking it and that the closure does not have to return any permissions. On line 18, we duplicate the pre-shared key (which the APPLICATION obtains as a program input) to keep the CORE’s memory footprint separated from the APPLICATION. Thus, we can pass half of the permissions for accessing the struct fields to the goroutine spawned on line 19 and store the remaining permissions in the invariant `Inv`, which is then returned to the caller of `InitChannel`.

APPLICATION. The APPLICATION (Fig. 3) consists of a single function that creates a closure that will print any incoming message, initializes the CORE with the pre-shared key `psk` and this closure, and then sends a message by invoking the `Send` function of the CORE. In realistic programs, the APPLICATION might have thousands of lines of code, making auto-active verification prohibitively expensive. We explain below how DIODON uses automatic static analyses instead.

Protocol model. Fig. 4 excerpts the abstract protocol model as a multiset rewriting system in Tamarin (cf. Sec. 3.1) with two protocol roles, Alice and Bob, each starting off with a pre-shared key `psk`. Both roles can send and receive unboundedly-many packets, each of which are the composition of a message plus the appropriate MAC. To make zero assumptions about the messages themselves, we treat them as being adversarially-controlled, i.e., the sending role obtains a message from the attacker-controlled network via an `In` fact, as shown on line 3. For this protocol model, we prove that all received messages were previously sent by either Alice or Bob, unless the attacker obtains the pre-shared key, which Tamarin proves automatically.

In order to prove that our program is actually a refinement of this model and, thus, inherits all proven properties, we combine auto-active verification and static analyses to obtain provably-sound guarantees.

Verification. Our goal is to ensure that the composition of the APPLICATION and CORE refines the abstract Tamarin model, i.e., the program’s I/O behavior is contained in the model’s I/O behavior. This refinement implies that any trace-based safety property proven in Tamarin also holds for the program because the program performs the same or fewer I/O operations than the protocol model. We split the refinement proof into three steps: We prove that (1) non-protocol I/O is independent from protocol secrets, (2) all remaining I/O refines a protocol role, and (3) the proof steps soundly compose.

First, we manually identify protocol-relevant calls to I/O operations within the CORE. In our example, these are the `sendToNetwork` call and the corresponding network receive operation. We then perform an automatic taint analysis on the entire codebase to prove I/O independence for all other calls to I/O operations (in our example, the calls to `Printf`), i.e., we check that they do not use tainted data. Uncommenting line 10 in Fig. 3 would result in printing all struct fields of variable `c` including the pre-shared key `psk`, which is the only secret. I/O independence would correctly fail for this modified program, resulting in an error message indicating the flow of secret data to the print statement. In general, we treat data as a secret (i.e., tainted) if the protocol model’s attacker might not know this data. Checking I/O independence ensures that we do not miss any protocol-relevant I/O operations and that the chosen CORE is sufficiently large.

The CORE may execute protocol-relevant operations not only by performing I/O operations, but also by communicating with the APPLICATION. For example, Alice’s protocol step of taking an arbitrary message from the environment

(before signing and sending it), is implemented by the CORE obtaining `msg` from the APPLICATION (line 25 in Fig. 2). Similarly, Alice may (after receiving a packet and checking its signature) release its payload to the environment, which is implemented as passing the payload to the APPLICATION when invoking the closure `c.cb` (not shown in Fig. 2). To handle such protocol-relevant operations uniformly, we treat them as *virtual* protocol-relevant I/O operations. This allows us to enforce or assume constraints on the arguments’ taint status while creating the necessary proof obligations in the next step of the refinement proof. Here, the fact that releasing the payload is permitted by the protocol model (line 10 in Fig. 4) informs the taint analysis that the callback’s argument may be considered as untainted, which allows printing it on line 6 in Fig. 3.

Second, we prove the CORE using the auto-active Gobra verifier. This proof includes showing that the protocol model permits every protocol-relevant I/O operation, including virtual I/O. Note that step (1) ensures that these operations must all be in the CORE. We use an I/O specification for each protocol role describing the permitted protocol-relevant I/O operations (cf. Sec. 3.3). In our example, Alice obtains the permissions to perform these operations during the initialization of the CORE (line 17) and maintains them as part of the invariant (line 34). When performing a protocol-relevant I/O operation, like `sendToNetwork`, Gobra proves that the I/O specification permits this operation with the specific arguments. Otherwise, Gobra reports a verification failure.

Third, since the CORE’s proof performed with Gobra assumes that callers respect the functions’ preconditions, we restrict the class of supported preconditions such that static analyses are able to prove that the APPLICATION satisfies them. For example, the precondition of `Send` requires exclusive access for the argument `msg`; we enforce this condition using a combination of static pointer and escape analyses to ensure that no other goroutine accesses the memory pointed to by `msg`. `Send`’s other precondition requires the CORE’s invariant to hold, which is established by `InitChannel`. The APPLICATION could in principle violate this precondition, for example, by creating a `Chan` instance without calling `InitChannel`, or by invalidating the invariant of a `Chan` instance through field updates or concurrency. Our combination of static analyses prevents all such cases (cf. Sec. 4.3).

Together, these three proof steps ensure that the program refines the abstract Tamarin model and inherits the security properties proved for the model.

3. Background

In this section, we provide the necessary background on the verification techniques that we reference in the rest of the paper. We detail verification of abstract protocol models (Sec. 3.1), verification of implementations (Sec. 3.2), and code-level refinement (Sec. 3.3), which transfers security properties from a protocol model to implementations.

3.1. Protocol Model Verification

We model a security protocol and prove security properties about it using Tamarin, an automated protocol model verifier. A protocol model consists of protocol roles and a DY attacker that are expressed as multiset rewrite rules. Each rule has the shape $L \multimap [A] \multimap R$, where L and R are multisets of facts and A is a set of actions. The system's state S is a multiset of facts, which is initially empty, and a rule can be applied if L is (multiset) included in S , i.e., $L \subseteq_m S$. Applying a rule removes the facts in L from and adds those in R to the system state, i.e., results in a new state $S \setminus_m L \cup_m R$. While most facts are user-defined to represent the state of a protocol role, Tamarin uses certain predefined facts. In particular, $\text{In}(x)$ and $\text{Out}(x)$ facts represent receiving and sending a message x from and to the attacker-controlled network, respectively. Since Tamarin uses equational theories to describe otherwise uninterpreted functions, such as $\text{dec}(\text{enc}(p, \text{pk}(sk)), sk) = p$ to describe asymmetric decryption (dec) w.r.t. asymmetric encryption (enc), Tamarin performs multiset inclusion modulo equational theories. A possible sequence of rule applications forms a trace that consists of each rule application's set of actions (A). Tamarin symbolically explores all possible traces involving unboundedly-many instances of protocol roles to prove a security property, or, if the proof fails, provides a trace of an attack as a counterexample to the security property.

Tamarin's symbolic DY attacker fully controls the network and can construct new messages by applying symbolic operations to terms it has observed. However, a symbolic attacker can neither perform arbitrary computations, nor can it exploit algorithmic weaknesses or side channels such as timing. Nevertheless, verification using Tamarin guarantees the absence of many relevant vulnerabilities and has proven effective in applications to 5G [7], EMV card protocols [24], and the Noise protocol family [25].

3.2. Code-Level Verification

We prove safety and functional properties of an implementation by reasoning about all possible executions statically, without any runtime checks. In this context, the term *safety* expresses that an implementation neither causes runtime exceptions nor undefined behavior. In particular, it covers the absence of memory errors, buffer overflows, and data races. *Functional properties* are implementation-specific and express the desired behavior, e.g., that a sorting algorithm's result is a sorted permutation of the input.

We perform *modular* verification, i.e., we reason about each function in an implementation in isolation. To do this, we equip a function with a *specification* that consists of a pre- and postcondition. A function's precondition is a logical formula specifying all valid program states in which this function can be called, and a function's postcondition specifies properties that hold for all valid program states after executing the function's body.

To reason about heap-manipulating programs, we use *separation logic* [23]. Separation logic allows us to express

precisely which heap fragment f a function operates on and provides connectives that split and combine heap fragments, e.g., when calling another function that operates only on a subfragment $f' \subseteq f$, we know that the function does not modify any heap location in the *frame* $f \setminus f'$.

Separation logic associates a *permission* with each heap location. A permission represents ownership of a particular heap location and is required for each access. We use fractional permissions [26] to distinguish between exclusive and shared ownership, which permits multiple threads in a concurrent program to simultaneously share ownership of a heap location. In specifications, we express permission to a heap location l with fraction p as $\text{acc}(l, p)$, cf. Sec. 2.

The *separating conjunction* $*$ is akin to regular conjunction, but requires the *sum* of the permissions in both conjuncts. For instance, $\text{acc}(l_1, 1) * \text{acc}(l_2, 1/2)$ specifies full and half permissions for heap locations l_1 and l_2 , respectively. Additionally, this example implicitly specifies that the heap locations are disjoint, i.e., $l_1 \neq l_2$. Otherwise, if l_1 and l_2 were aliased, the permission amounts would add up to $3/2$ contradicting separation logic's invariant that at most a full permission exists for a heap location. In our code listings, we overload $\&\&$ to denote separating conjunction.

We use separation logic *predicates* [27] to abstract over individual permissions to heap locations as demonstrated by the CORE's invariant in our running example. Conceptually, we can treat a predicate instance such as $\text{Inv}(c)$ in Fig. 2 as a shorthand notation for the predicate's body.

Proving that each function has sufficient permissions for each heap access guarantees safety. E.g., a buffer overflow corresponds to accessing an array element out of bounds; this is prevented since allocating an array creates permissions only for in-bound elements. Similarly, data races are prevented since two threads simultaneously writing the same heap location would require that each thread has write permission for this heap location, which is impossible as there is only a single write permission for any given heap location.

Various separation logic-based program verifiers exist, including VeriFast [28] and VST [29] for C, Gobra [21] for Go, Nagini [30] for Python, and Prusti [31] for Rust. These verifiers are *auto-active*, i.e., use manual annotations and proof automation to prove properties about programs.

3.3. Code-Level Refinement

Arquint et al. [14] split verification of security protocol implementations into two steps: proving security properties for an abstract protocol model using Tamarin, and using an auto-active program verifier to prove that an implementation refines this model. This approach disentangles proving global security properties from local reasoning about implementations, while exploiting each tool's automation.

To connect these two steps, Tamarin automatically generates an *I/O specification* for each protocol role in a given abstract protocol model. A protocol role's I/O specification describes the permitted I/O operations including their sequential ordering and arguments. By verifying that an implementation executes at most the operations permitted

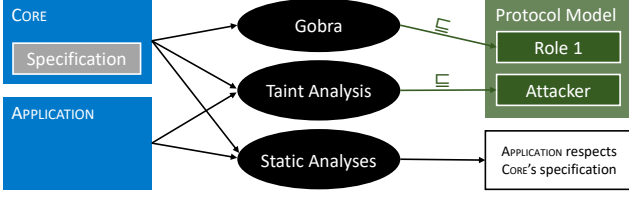


Figure 5. **DIODON** proves that the entire codebase refines a protocol model by soundly composing auto-active verification with automatic static analyses. We auto-actively verify the **CORE** based on its specification using **Gobra** to show that the protocol-relevant I/O operations refine a protocol role. The static taint analysis proves that all other I/O operations within the entire codebase refine our attacker model. Lastly, we discharge **Gobra**’s assumptions by applying automatic static analyses, proving that the **APPLICATION** satisfies the calling restrictions expressed in the **CORE**’s specification.

by the I/O specification, we prove that all executions of this implementation result in a trace of I/O operations that is included in the set of traces considered by Tamarin when verifying the security properties. Thus, the implementation satisfies the same security properties as the model.

The I/O specification is expressed in I/O separation logic [32], a dialect of separation logic, which is readily supported by separation logic-based verifiers. I/O separation logic extends the use of permissions beyond guarding heap accesses to guarding I/O operations by associating an *I/O permission* with each I/O operation. We equip library functions performing I/O operations with a specification that checks and consumes the corresponding I/O permission.

Successful code-level verification against the library functions equipped with I/O permissions guarantees that an implementation performs at most the I/O operations specified in the Tamarin model and respects their sequential ordering. Otherwise, verification fails because a prohibited I/O operation would require an unavailable I/O permission.

4. DIODON

Our methodology, **DIODON**, proves security properties for implementations by refinement and scales to large codebases by significantly reducing verification effort. **DIODON** enables more concise protocol models than previous approaches and leverages fully automatic analyses on most of the implementation to discharge proof obligations.

We manually decompose a codebase into two syntactically isolated components, the **CORE** implementing a security protocol, and the **APPLICATION** consisting of the remaining code. Typically, this decomposition is natural and follows module boundaries as a protocol’s implementation is localized. As illustrated in Fig. 5, this decomposition allows us to split the proof that the entire codebase refines a protocol model into three steps and uses the best-suited tool for each step. We explain in Sec. 4.1 how **DIODON** identifies which I/O operations are protocol-relevant by performing a static taint analysis on the entire codebase. Sec. 4.2 covers the **CORE**’s auto-active verification using **Gobra** proving that protocol-relevant I/O operations refine a particular protocol role. Finally, Sec. 4.3 explains how we discharge

the assumptions made when auto-actively verifying the **CORE** by performing static analyses on the **APPLICATION**.

4.1. I/O Independence

One of our key insights is to distinguish between I/O operations that are relevant for a security protocol from those that are not (e.g., sending log messages to a remote server). This distinction has two main benefits. First, protocol-irrelevant operations do not have to be reflected in the abstract protocol model, which makes the model concise, more general, and easier to maintain, review, and prove secure. Second, by ensuring that protocol-irrelevant I/O operations cannot possibly invalidate the security properties proven for the protocol model, we do not have to consider them during the laborious auto-active refinement proof and instead can check simpler properties using automatic static analyses. We classify all calls to I/O operations as either protocol-relevant or protocol-irrelevant. In the **CORE**, an I/O operation is protocol-irrelevant if and only if its specification requires no I/O permissions. In contrast, all I/O operations in the **APPLICATION** are implicitly considered protocol-irrelevant.

To ensure that I/O operations classified as protocol-irrelevant indeed do not interfere with the protocol or invalidate proven security properties of the protocol, we check that they do not use any secret data (such as key material); more precisely, we check non-interference between protocol secrets and the parameters of these I/O operations. We call this important property of an I/O operation *I/O independence*. It guarantees that an I/O operation cannot possibly invalidate the protocol’s proven security properties: any I/O operation that uses only non-secret data could also have been performed by the **DY** attacker and, thus, was already considered by Tamarin during the protocol verification. In other words, proving that all protocol-irrelevant I/O operations satisfy I/O independence guarantees that they refine our **DY** attacker (cf. App. B.1).

We prove I/O independence by performing an automatic static taint analysis on the entire codebase. A taint analysis checks for a set of sources and sinks whether there are any flows of information from a source to a sink. The analysis starts at each source, i.e., a function which produces secret data, and explores how secret information propagates through the program by keeping track of program locations storing a *tainted* value, i.e., a value that is influenced by a source. We disallow branching on tainted data to avoid information flows via control flow.

We configure the taint analysis to consider all calls to key-generation functions within the **CORE** and long-term secrets that are passed as program inputs, like the pre-shared key in our running example, as sources because the **DY** attacker does not have access to them. This set of initial sources taints all protocol secrets including session keys. E.g., if the **CORE** implements a DH key exchange, the analysis correctly considers the generated secret key and the resulting shared key tainted because the shared key is computed from the secret key and the other party’s public key. We then configure the taint analysis to treat all I/O operations

in the APPLICATION as well as all protocol-irrelevant I/O operations in the CORE as sinks. We use Capslock [33] to identify such I/O performing functions in the Go standard library. We consider all functions with at least one of the following capabilities as a sink: write to the file system or network, modify the state of the operating system (e.g., `os.Setenv`), perform a system call, and execute external programs (e.g., `(*os/exec.Cmd).Run`).

We run the taint analysis on the entire codebase. If taint reaches a sink, verification fails because a secret reached a supposedly protocol-irrelevant I/O operation. Otherwise, we have correctly identified the protocol-relevant I/O operations (and thereby confirmed that we have correctly delimited the CORE); it remains to reason about those I/O operations, as we discuss next.

4.2. CORE Refinement

We auto-actively verify the entire CORE, which allows us to state and prove (besides safety and functional correctness) precise constraints about protocol-relevant calls to I/O functions and their arguments. In particular, we prove that the implementation uses the payload for each I/O operation specified in the protocol model. The corresponding verification effort is feasible since, in industrial codebases like our main case study, the CORE comprises only a small fraction of the codebase.

We prove that the CORE refines a protocol role by building on the approach explained in Sec. 3.3. In particular, we equip each protocol-relevant I/O operation with a specification that requires an I/O permission for executing this operation with the provided arguments. Since we provide exactly the I/O permissions justified by the protocol role's model to the CORE during its initialization, successful verification with Gobra implies that the CORE executes at most the protocol-relevant I/O operations permitted by the model and, thus, refines this protocol role. This approach is inspired by Arquint et al. [14], but differs in three significant ways.

First, we do not auto-actively verify the entire codebase and, instead, verify only the CORE. As we will discuss in Sec. 4.3, we syntactically restrict the preconditions of the CORE functions so that we can apply automatic static analyses to check that each call from the APPLICATION satisfies them, which is necessary for soundness.

Second, our approach supports codebases that use multiple instances of the CORE, e.g., to run multiple roles of the protocol or to run the protocol multiple times. Since Tamarin considers unboundedly-many role instantiations, we can soundly create the required I/O permissions for executing a role instance whenever we create a new CORE instance. These I/O permissions are bound to an instance's unique identifier such that each CORE instance has its own set of I/O permissions for executing the security protocol once.

Third, to reflect that interactions in the model between the protocol and the environment may manifest as interactions between the CORE and the APPLICATION in the implementation, we treat the boundary between them as a virtual

```

1 //@ pres c ≠ nil ⇒ inv(c)
2 //@ pres a0 ≠ nil ⇒ acc(a0)
3 //@ pres a1 ≠ nil ⇒ acc(a1)
4 //@ ens r ≠ nil ⇒ acc(r)
5 func (c *Core) ApiFn(a0, a1 *int) (r *int)

```

Figure 6. Example of a signature and specification of a CORE API function.

network interface and enforce I/O permissions for the corresponding virtual I/O operations, as we illustrated in Sec. 2.

4.3. Analyzing the APPLICATION

We now show how to scale auto-active verification to the entire codebase. Applying auto-active verification to an entire codebase is typically not feasible within the resource constraints of industrial projects. A key insight of DIODON is that this is not necessary: we can use static analyses to automatically discharge separation logic proof obligations arising in the APPLICATION to obtain, together with the verified CORE, a proof in separation logic for the *entire* codebase.

The refinement proof for the CORE is valid in the context of the entire application if (1) each call to a CORE function from the APPLICATION satisfies the function precondition, and (2) the APPLICATION respects permissions on memory accesses. Our soundness proof for DIODON (cf. App. B.2) ensures that these proof obligations are sufficient and that our novel combination of static analyses can soundly discharge them. We illustrate these proof obligations and how we discharge them by considering the exemplary CORE function in Fig. 6, taking two integer pointers as input and returning an integer pointer. This function maintains the CORE invariant (if `c` is non-`nil`), needs full permissions for both inputs, and returns full permissions for the input and output parameters (if they are non-`nil`). Thus, we cannot allow, e.g., the APPLICATION to pass two aliased arguments (cf. Sec. 3.2) to this function or to concurrently access heap locations pointed to by these arguments as this would violate the precondition.

Implicit annotations. To construct a proof for the entire codebase, we enrich the APPLICATION with a *hypothetical program instrumentation* that connects the APPLICATION to the necessary proof obligations imposed by the proof of the CORE. These *implicit annotations* track the permissions that the APPLICATION owns by using sets of heap locations and a *program invariant* specifying permissions for the heap locations in these sets. More precisely, each thread has a set `slh` for thread-local objects such as buffers and a set `sih` for CORE instances. Similarly, a global set keeps track of objects that might be shared between threads, which becomes relevant later. The sets `slh` and `sih` allow us to state the following local program invariant that must hold at every program point in the APPLICATION.

$$\Pi_l \triangleq (\star_{l \in \text{slh}} \text{acc}(l)) \star (\star_{i \in \text{sih}} \text{inv}(i))$$

Here, the iterated separating conjunction $\star_{e \in s} a(e)$ conjoins the assertions $a(e)$ using separating conjunction for all elements e in set s . Π_l states that a thread holds full permissions

```

1  //@ assert (★l ∈ slh acc(l)) * (★i ∈ sih inv(i))
2  //@ sih := sih \ {c}
3  //@ assert (★l ∈ slh acc(l)) * (★i ∈ sih inv(i)) *
4  //@ (c ≠ nil ⇒ inv(c))
5  //@ slh := slh \ {a0}
6  //@ assert (★l ∈ slh acc(l)) * (★i ∈ sih inv(i)) *
7  //@ (c ≠ nil ⇒ inv(c)) *
8  //@ (a0 ≠ nil ⇒ acc(a0))
9  //@ slh := slh \ {a1}
10 //@ assert (★l ∈ slh acc(l)) * (★i ∈ sih inv(i)) *
11 //@ (c ≠ nil ⇒ inv(c)) *
12 //@ (a0 ≠ nil ⇒ acc(a0)) *
13 //@ (a1 ≠ nil ⇒ acc(a1))
14 r := c.ApiFn(a0, a1)
15 //@ assert (★l ∈ slh acc(l)) * (★i ∈ sih inv(i)) *
16 //@ (c ≠ nil ⇒ inv(c)) *
17 //@ (a0 ≠ nil ⇒ acc(a0)) *
18 //@ (a1 ≠ nil ⇒ acc(a1)) *
19 //@ (r ≠ nil ⇒ acc(r))
20 //@ slh := slh ∪ ({a0, a1, r} \ nil)
21 //@ sih := sih ∪ ({c} \ nil)
22 //@ assert (★l ∈ slh acc(l)) * (★i ∈ sih inv(i))

```

Figure 7. Conceptually inserted implicit annotations for a CORE API call $r := c.\text{ApiFn}(a0, a1)$ in the APPLICATION. The assert statements solely illustrate our deductions and, thus, can be omitted.

for all objects in slh and the CORE invariant for all instances in sih . In addition, these permissions are disjoint allowing the APPLICATION to write to heap locations in slh without breaking the CORE invariant. When a thread obtains or gives up permissions, our implicit annotations adjust slh and sih to maintain the program invariant.

Fig. 7 shows these *implicit annotations* for calls to ApiFn . To highlight that each statement in the APPLICATION maintains the program invariant, we assert Π_l on lines 1 and 22. For each permission required by the callee’s precondition, we remove the corresponding heap location from one of the sets to reflect that ownership is being passed to the callee. Assuming (for now) that the location was originally in the set, this removal extracts the corresponding permission from Π_l , as illustrated by the intermediate assert statements starting on lines 3, 6, and 10 for the three arguments of the call. After the call, we conversely add those heap locations to the sets for which the callee’s postcondition provides permissions.

For each permission in the precondition, if the corresponding heap location was contained in one of the sets before the removal operation, then we have effectively proved that the precondition holds (syntactic restrictions ensure that the preconditions cannot contain constraints other than permission requirements, see below). In the rest of this subsection, we explain how we use static analyses to check this set containment. Then, we explain the proof obligations for memory accesses within the APPLICATION.

Guaranteeing permissions for parameters. For the arguments $a0$ and $a1$ (we will discuss the core instance c below), we need to prove that (1) $\{a0, a1\} \subseteq slh$ holds before the call to ApiFn and (2) $a0$ and $a1$ do not alias. If (2) was violated, $a1$ would no longer be in slh after removing $a0$ on line 5 in Fig. 7, i.e., we would obtain only

Condition	Details
C1 CORE init	CORE instances are created in a function ensuring the invariant in its postcondition
C2 No modification	APPLICATION does <i>not</i> write to CORE instances’ internal state, even through an alias
C3 CORE preservation	CORE instances are passed only to CORE functions that preserve the invariant
C4 CORE locality	CORE instances are used only in the thread they are created in
C5 CORE callback	CORE APIs are not invoked in APPLICATION callbacks
C6 Parameter locality	Parameters to CORE APIs are local
C7 Disjoint parameters	Parameters to the same CORE API call do not alias one another
C8 APPLICATION access	Reads and writes in the APPLICATION occur to memory allocated in the APPLICATION or transferred from the CORE

Figure 8. Sufficient conditions checked by our static analyses, grouped into those involving CORE instances, other parameters to CORE functions, and memory accesses in the APPLICATION.

$\text{acc}(a0)$ instead of $\text{acc}(a0) * \text{acc}(a1)$.

We discharge these two proof obligations by checking the conditions (C6) and (C7) in Fig. 8, respectively, using static analyses. We check (C6) by using a thread escape analysis, which delivers judgments $\text{local}(x)$ for a particular program point expressing that $*x$ is definitely not accessible by any other thread. We show in App. B.2 that (C6) suffices to discharge $\{a0, a1\} \subseteq slh$ (if the arguments are non- nil) by proving a lemma that relates $\text{local}(x)$ for a program point p with $x \in slh$. We obtain (C7) by applying a pointer analysis, which computes may-alias information, i.e., $\text{pts}(x)$ for a pointer x , where $a \in \text{pts}(x)$ denotes that $*x$ may-alias any location allocated at site a . More precisely, we check for each pair of arguments that the sets of locations they may-alias are disjoint, which is sufficient as we restrict parameters to be shallow.

Guaranteeing the CORE invariant. Similarly to parameters, we have to prove that $c \in sih$ holds such that removing c from sih on line 2 grants us the CORE invariant $\text{inv}(c)$, if c is non- nil . In App. B.2, we prove that $c \in sih$ if the following premises hold. (1) The CORE instance c must have been returned as a result from a CORE API function initially establishing the CORE invariant, e.g., InitChannel in our running example. (2) All heap modifications in the APPLICATION must not modify the internal state of the CORE instance, even through an alias, since this could invalidate the CORE invariant.

In a single-threaded program without callbacks from the CORE to the APPLICATION, the above premises are sufficient. However, in the presence of these two features, we need to ensure that the APPLICATION does not call two CORE functions on the same CORE instance simultaneously, which would effectively duplicate permissions and, thus, make reasoning unsound: (3) The APPLICATION must not pass the same CORE reference to more than one thread, and (4) the APPLICATION must not call a CORE function in a callback on the same instance that is invoking the callback.

We establish the four premises by checking the conditions (C1) to (C5) in Fig. 8. Conditions (C1) and (C3) can be enforced by checking that the APPLICATION calls only CORE functions that establish or preserve the invariant. While the postconditions provide this information for CORE instances that are passed as arguments or results, our analyses need to prevent a subtle loophole: We need to prevent CORE functions from allocating a CORE instance *without* establishing its invariant and letting the APPLICATION access it via global variables or shared memory. We implemented a pass-through analysis computing $\text{pass}_f(x, r)$ for a function f stating that outside of calls to f , $*x$ definitely passed through return parameter r . We use this pass-through analysis to ensure that all references to CORE instances in the APPLICATION are obtained exclusively through the return parameter, such that the postcondition establishes the invariant.

To establish (C2), we use a pointer analysis to ensure that all reads and writes in the APPLICATION never access a CORE instance’s internal state. In particular, we ensure that the APPLICATION accesses *only* heap locations that must-not-alias locations reachable from sih , i.e., internal state of CORE instances. Since we use a sound pointer analysis, this check conservatively over-approximates the heap locations about which the CORE invariant states properties. While it is possible to access CORE memory without breaking the invariant, we could not treat the CORE invariant as an opaque separation logic resource when analyzing the APPLICATION, which would require a static analysis capable of reasoning about fractional permissions and arbitrary functional properties.

For (C4), we use the thread escape analysis to ensure that each CORE instance does not escape its thread (we show $\text{local}(c)$ for each call to CORE instance c), guaranteeing that each thread operates on a disjoint set of CORE instances sih . While it is possible to safely pass CORE instances between threads, this would require a significantly more sophisticated static analysis that can reason about the ordering of concurrent executions. Condition (C5) is enforced by checking that the callgraph does not contain CORE functions invoked transitively from APPLICATION callbacks. Allowing such calls would require proving that the same instance is not used in the inner call, which requires a more precise pointer analysis.

Our explanations generalize to CORE API functions as long as they satisfy the following restrictions on pre- and postconditions. We support an arbitrary number of input and output parameters with arbitrary value and pointer types. Our restrictions mandate that CORE API functions preserve the CORE invariant and full permissions for each parameter of pointer type, both only under the condition that the receiver and parameters are non-**nil**. Additionally, the postcondition specifies full permissions for each return parameter if it is non-**nil** and of pointer type. These restrictions ensure that preconditions do not specify functional properties, such as require an input array to have a certain length, which we cannot check using our static analyses. As seen with our example in Fig. 6, we cannot rule out that the APPLICATION passes **nil** as an argument because there is no sound nilness analysis for Go to the best of our knowledge and, thus, we account for this possibility in our restrictions.

APPLICATION memory access. Finally, we need to ensure that the APPLICATION accesses only memory to which it has permissions. While we have already established that the APPLICATION does not write to internal state of CORE instances (C2), we need to particularly consider the case where memory is transferred after its allocation between the CORE and the APPLICATION. The other case, namely the CORE or APPLICATION allocating memory without transferring it, is straightforward. I.e., if CORE-allocated memory is never transferred to the APPLICATION then the APPLICATION cannot access it. Similarly, if APPLICATION-allocated memory is not transferred to the CORE then the APPLICATION retains the corresponding permission.

Checking condition (C8) is sufficient. If APPLICATION-allocated memory is transferred to the CORE, our syntactic restrictions guarantee that the CORE only temporarily borrows the corresponding permissions until the CORE API call returns. If the CORE allocates memory and transfers it to the APPLICATION, the CORE must also transfer the corresponding permissions, which we enforce via our pass-through analysis checking that this transfer happens via a return parameter as our syntactic restrictions guarantee that the postcondition specifies permissions for this return parameter. Using (C8), we prove that each memory access in the APPLICATION is to a location in either slh or sgh (cf. App. B.2). In the latter case, we need to reason about concurrent access. We assume that the APPLICATION is free from data races: if two accesses race, then the program is invalid according to the Go specification. If there are no races, then there is some total order in which the threads can atomically pull permission from sgh , perform the access, and then return permissions to sgh before the next thread needs to access the same location.

4.4. Threat Model and Assumptions

DIODON enables proving strong security properties for large codebases with manageable proof effort, but, like other verification techniques, relies on assumptions about the codebase, execution environment, and the employed tools.

DIODON considers an arbitrary number of potentially concurrent protocol sessions, allowing the DY attacker to, e.g., replay messages across sessions or apply cryptographic operations thereto to construct messages of unbounded size. As is standard for symbolic cryptography, we assume cryptographic operations such as signing are perfectly secure, e.g., the attacker can create valid signatures only if it possesses the correct signing key. The attacker can obtain such keys only by observing or constructing them, never by guessing.

Our methodology allows us to prove that each implementation individually refines a particular role of an abstract protocol model. Since the security properties we prove about an abstract model are typically global, they hold only if each involved implementation refines one of the protocol roles.

DIODON uses several tools to discharge proof obligations, and we rely on the soundness of each tool: the abstract protocol model verifier, the auto-active program verifier, and the static analyses. The risk that any of these tools is

unsound can be mitigated by choosing mature tools such as Tamarin and Gobra.

More specifically, the CORE’s auto-active verification relies on trusted specifications for libraries, such as the I/O or cryptographic libraries that, e.g., consume I/O permissions or specify the cryptographic relations between input and output parameters. DIODON could be combined with verified libraries like EverCrypt [34] to reduce this trust assumption.

Furthermore, our taint analysis relies on the correct specification of secrets and I/O operations (we use an existing tool [33] to identify I/O operations). E.g., not treating the pre-shared key in the running example as a secret would allow us to perform I/O operations that depend on this key in the APPLICATION.

The employed static analyses assume that the entire codebase is free of data races. While we auto-actively prove race freedom for the CORE, this remains an assumption for the APPLICATION. Our implicit annotations clearly indicate where in the APPLICATION we rely on this assumption. Additionally, the static analyses do not soundly handle certain hard-to-analyze features such as the `unsafe` package (e.g., allowing arbitrary pointer arithmetic), `cgo` (i.e., the ability to invoke C functions), or reflection. We rely on the codebase not using them in a way that would invalidate the analysis results. DIODON could be extended by additional static analyses to reduce these assumptions, e.g., by performing a data race analysis and checking for uses of the `unsafe` and `cgo` packages and reflection. As such, these assumptions are not an inherent limitation of DIODON itself.

5. Case Studies

To demonstrate that DIODON scales to large codebases, we evaluate it on the AWS Systems Manager Agent (SSM Agent) [20], a 100k+ LoC production Go codebase. Furthermore, we apply DIODON to a small implementation of the signed Diffie-Hellman key exchange to showcase that our approach applies to other implementations and coding styles.

5.1. AWS Systems Manager Agent

The AWS Systems Manager Agent (SSM Agent) [20] provides features for configuring, updating, and managing Amazon EC2 instances, and is widely used by AWS customers. A part of this codebase implements a novel protocol which enables encrypted interactive shell sessions with remote host machines, similar to SSH, without needing to open inbound ports or manage SSH keys. This protocol establishes these shell sessions with a handshake protocol involving a signed elliptic-curve Diffie-Hellman key exchange to derive sessions keys that are subsequently used in the transport phase to encrypt the shell commands and their results.

We apply DIODON by first modeling the protocol in Tamarin and proving secrecy and injective agreement (Sec. 5.1.1). Second, we partition the codebase into the code implementing the protocol (the CORE) and the remaining codebase (the APPLICATION), and prove I/O independence (Sec. 5.1.2). Third, we auto-actively verify the CORE

	Tool	Proof Effort	Execution Time
Protocol Model	Tamarin	<2 pms	3.18 min
CORE Refinement	Gobra	<3 pms	1.26 min
I/O Independence	Argot	<0.5 pm	0.58 min
CORE Assumptions	Argot	<1.5 pms	2.51 min

Figure 9. Execution time for running each tool on the SSM Agent codebase and approximate proof effort in person-months (pms) for creating a protocol model, adding specifications, and adapting the Argot analyses, respectively.

using Gobra to prove that the CORE refines the SSM Agent’s role (Sec. 5.1.3). Finally, we apply the automatic static analyses Argot [22] to discharge the assumptions within the APPLICATION on which the auto-active proof relies (Sec. 5.1.4).

Fig. 9 overviews each tool’s execution time, for which we use the 10 % Winsorized mean of the wall-clock runtime across 10 verification runs, measured on a 2023 Apple MacBook Pro with M3 Pro processor and macOS 15.3.2.

5.1.1. Protocol model. We model in Tamarin the security protocol for establishing a remote shell session between an SSM Agent running on an EC2 instance and an AWS customer. The protocol offloads all signature operations to the AWS Key Management Service (KMS) [35] such that neither protocol role has to manage their own signing keys. We model the connections to KMS as secure channels. Furthermore, the SSM Agent sends the asymmetrically-encrypted session keys to a trusted third party to monitor the transmitted shell commands should this be necessary for regulatory reasons. For space reasons, we provide the full description of the protocol online App. A.

In Tamarin, we prove secrecy for the two symmetric session keys, i.e., the attacker does not learn these keys unless the SSM Agent’s or customer’s signing key or the monitor’s secret key is corrupted. Additionally, we prove that the SSM Agent injectively agrees with the customer, and vice versa, on their identities and the session keys, unless one of the three aforementioned corruption cases occurs.

The abstract protocol model amounts to 341 lines of code and is automatically verified by Tamarin 1.10.0 in 3.18 min using an auxiliary oracle consisting of 75 lines of Python code.

5.1.2. Proving I/O independence. We perform a taint analysis to prove I/O independence. We configure the taint analysis to consider all generated elliptic-curve secret keys as sources of protocol secrets. We assume that only the CORE uses the SSM Agent’s signing keys and do not treat KMS responses as taint sources because KMS only sends us signatures and never key material. As described in Sec. 4.1, we use Capslock’s capability information to automatically configure the taint analysis’ sinks.

We annotated some branching operations, instructing the taint analysis to ignore that the condition is tainted. We identified two classes of such branching operations. The first class is justified by cryptography. E.g., we allow branching on the success of decrypting a transport message because leakage is minimal. The second class results from imprecisions of the taint analysis and corresponds to false posi-

tives, i.e., the analysis deems the branch conditions tainted although it is not. To avoid another source of false positives, we configured the taint analysis to ignore taint escaping the current thread (which would otherwise always lead to errors). Such cases could be handled precisely by marking certain struct fields as potentially storing concurrently-accessed, tainted data, such that the analysis can track the taint.

The taint analysis succeeds for the SSM Agent codebase in 0.58 min, proving that there are no taint flows.

5.1.3. CORE refinement. The SSM Agent contains a Go package called `datachannel` that implements the protocol. More precisely, this package contains struct definitions that together store all necessary internal state. Additionally, this package exposes publicly accessible functions to initialize the internal state, perform a handshake, and send a payload, which internally rely on several private functions. We refer to these struct definitions and functions as the CORE. For backward compatibility, the CORE also implements a legacy protocol; we assume that this legacy protocol is disabled.

Implementation. Each CORE instance corresponds to one run of the protocol with a particular customer. During initialization of a new CORE instance, the CORE starts a new thread, responsible for receiving and processing incoming packets for this protocol run, similar to the running example. If an incoming packet contains a transport phase payload, this payload is delivered by a callback to the APPLICATION. Thus, the CORE uses two different threads, one for sending messages and another one for receiving messages, which both operate on shared state. This shared state keeps track of the progress within the protocol and the secret data involved in the protocol, such as the elliptic-curve Diffie-Hellman points and the resulting session keys.

Since the shared state is modified during the handshake, accesses must be synchronized to avoid data races. Hence, the CORE employs Go channels, i.e., lightweight message passing, to signal a transfer of the shared state’s ownership from one thread to another. During the handshake phase, exclusive ownership is transferred such that the threads have synchronized write access to the shared state. Afterwards, the shared state, which includes the established session keys, is used in a read-only way permitting both threads to concurrently read the shared state while sending and receiving transport messages.

Auto-active refinement proof. We verify the CORE using Gobra, which proves that the CORE refines the Tamarin model’s SSM Agent role. This proof encompasses memory safety, i.e., we prove that the CORE has sufficient permissions for every heap access, thus, guaranteeing absence of data races. In particular, this forces us to reason precisely about the shared state accesses that the two threads within the CORE perform.

Due to the intricate interplay of these threads, the resulting memory safety proof is substantial and requires Gobra’s expressivity. We isolate and axiomatize operations that Gobra does not yet support such as simultaneously receiving on multiple channels and functionally reasoning

about serialization and deserialization. For the purpose of the proof, we treat the CORE as a state machine consisting of 12 different states. This allows us to refer to these states in the CORE’s invariant and precisely express for each state the permissions and progress w.r.t. the abstract protocol model.

Although the entire complexity of the proof is encapsulated in the CORE’s invariant, function calls to the CORE must respect its state machine. To avoid exposing the state machine in these functions’ preconditions and imposing additional restrictions on callers, we slightly changed the implementation to perform a dynamic check consisting of a comparison with `nil` and a single integer comparison ensuring that the state machine is in a correct state; otherwise, these CORE functions return a descriptive error. Thus, the CORE functions’ specifications are similar to those of our running example, i.e., mention only the invariant and specify permissions for parameters without referring to the state machine. While most parameters are of primitive type or shallow, there are a few non-shallow input parameters, which the CORE treats as opaque. Similarly, the callback from the CORE to the APPLICATION delivers a non-shallow struct for which we ensure that the CORE passes permissions for all transitively reachable heap locations to the APPLICATION.

We prove memory safety and refinement of the CORE in 1.26 min for 750 lines of code requiring 3825 lines of specification and proof annotations; 1064 thereof are related to the I/O specification and generated automatically by Tamarin.

5.1.4. Analyzing the APPLICATION. The auto-active proof for the CORE relies on callers satisfying the specified preconditions, which we establish using a combination of static analyses. We implemented automatic checks as described in Sec. 4.3 for conditions (C1)–(C4) and (C6)–(C8). Condition (C5) requires a more precise call graph than is currently available in our tool and is, thus, left as future work.

We implemented our analyses by forking and extending the existing Argot tool. Most of our analyses are obtained by interpreting the output of an existing analysis; e.g., the parameter alias check uses the off-the-shelf pointer analysis to show parameters do not alias one another.

For some conditions, our static analyses were not able to validate the APPLICATION due to tool limitations. For example, the escape analysis cannot reason about which fields are accessed after a struct escapes. This can cause the tool to raise alarms when a struct stores a CORE instance in a field. We found it was straightforward to rewrite the CORE and APPLICATION to eliminate these failures. For example, the struct leakage can be fixed by moving the relevant field accesses before thread creation, so that the new thread has access only to the values of those fields and not the entire struct, and by extension the CORE instance.

By running our escape analysis, we observed that CORE instances escape the thread in which they are created because the APPLICATION creates a closure that closes over an object that points to a CORE instance. This capture is incidental in that the closure does not access the captured CORE instance, which we verified by manual inspection. This capture can

be eliminated by rewriting the application to only reference the state necessary in this closure, rather than the full object. This change would result in a more defensive implementation by reducing the scope of possibly concurrent accesses.

Our pass-through analysis is a prototype that succeeds on our second case study. However, for the SSM Agent, we obtain false positives due to allocations in functions called from both CORE and APPLICATION, which could be addressed by adding calling context information.

Some CORE functions take a pointer to a logger object as a parameter, which is internally thread-safe and shared between threads. We can safely ignore escape errors due to these parameters because the CORE does not access any memory of the logger object; the pointer is just used as an opaque reference to invoke log functions that are part of the APPLICATION.

In summary, this case study demonstrates that DIODON allows one to obtain strong security guarantees for a production codebase that was not designed with formal verification in mind. The remaining limitations (manual overrides of false positives in the static analyses, checking condition (C5), extremely lightweight dynamic checks enforcing non-nilness and correct ordering of API calls, and minor code changes) are modest compared to the complexity of the overall verification challenge and we conjecture that we can lift them by employing more precise static analyses.

5.2. Signed Diffie-Hellman Key Exchange

We also apply our approach to a codebase employing inverted I/O, i.e., has a CORE that only produces and consumes byte arrays corresponding to protocol messages while the APPLICATION performs all I/O operations. We adapted the Tamarin model and Go implementation of the signed Diffie-Hellman key exchange from Arquint et al. [14] and extended both by a transport phase that uses the established session key to send and receive unboundedly many payloads. Tamarin verifies the abstract model with 198 lines of code in 3.3 s while Gobra verifies the CORE consisting of 178 lines of code in 14.7 s. Executing all static analyses including the taint analysis takes 10.8 s.

This case study clearly exhibits the concept of virtual I/O. The CORE performs a virtual input operation for messages that the APPLICATION received from the network and forwarded to the CORE. Similarly, we perform a virtual output operation for every message that the CORE produces before returning this message to the APPLICATION. Therefore, we prove that the Tamarin model permits sending this message and in return, we sanitize the message from a taint analysis’ perspective such that the APPLICATION can send the message without causing a false-positive taint flow.

DIODON separates the justification of sending a particular message from the actual I/O operation. This is important for tackling realistic codebases because identifying the actual send operation in a call stack is typically difficult as a message passes through several functions that, e.g., add additional protocol headers before a message is handed to the network interface controller.

5.3. Discussion

Our evaluation demonstrates that DIODON enables us to efficiently prove that an entire codebase refines a protocol model and therefore is secure. To obtain the security properties as proven in Tamarin for a deployment of this protocol, we have to prove the implementations of all other protocol roles analogously against the same model using DIODON.

As shown in Fig. 9, the efforts for applying DIODON to the SSM Agent is manageable. Thanks to I/O independence, the Tamarin model is concise and can focus on the relevant interactions between the protocol roles. In addition, I/O independence allows us to apply automatic static analyses at the code-level to reason about all protocol-irrelevant I/O operations. This contrasts existing approaches that would auto-actively verify the entire codebase and prove that every I/O operation is explicitly permitted by the model, which is completely impractical for this codebase.

To evaluate DIODON’s effectiveness at preventing security vulnerabilities, we deliberately introduce bugs in our case studies. E.g., our taint analysis correctly fails if the CORE’s internal state, which includes the established session keys, is logged after the handshake. Additionally, sending the secret key in plaintext correctly results in Gobra failing to prove refinement w.r.t. the abstract protocol model. The tools’ execution time in the presence of these bugs remains comparable to that for the secure implementations.

By applying DIODON we not only obtain security properties for the SSM Agent codebase but we also discovered and fixed bugs along the way. Tamarin allowed us to quickly locate and fix a person-in-the-middle (PITM) attack in an earlier and unreleased version of the protocol, which is possible if the intended recipient’s identity is omitted in the signatures (sig_x and sig_y in App. A). On the code level, we identified and fixed a potential data race in an earlier and unreleased version of the CORE caused by insufficient synchronization between the two threads that send and receive handshake messages. We uncovered this data race because completing the memory safety proof for the CORE’s earlier version is not possible as an additional synchronization point is necessary to transfer separation logic permissions between these threads. This demonstrates the power of applying formal methods because detecting this data race with testing techniques would require to precisely time the reception of a handshake message such that the faulty memory access occurs and, thus, can be observed.

6. Related Work

Much prior work on verifying security protocols exists and surveys [36], [37], [38] provide an extensive overview. Hence, we focus on approaches for verifying security properties for *implementations* and their applicability to large and real-world codebases. We end by comparing DIODON to approaches based on dynamic verification.

Implementation and model generation. One approach to obtain verified protocol implementations generates secure-by-construction implementations from an abstract model,

e.g., [39], [40], [41], [42]. However, these implementations typically show subpar performance and optimizing them by hand or integrating them into a larger codebase forfeits proven security properties. Thus, the abstract model has to cover the entire functionality (which we do not require).

An alternative approach extracts an abstract model from an implementation, e.g., [4], [6], [43], [44], [45]. However, for this extraction to work, an implementation typically has to follow restrictive coding disciplines such that relevant protocol steps can be identified and extracted. To achieve isolation between a verified component and potentially malicious code, Kobeissi et al. [6] build on process isolation provided by operating systems and, thus, require verifying the entire critical process. We cannot adopt this approach because it requires changing the codebase heavily to split it into several processes and results in an, for our use case, unacceptable overhead, since each process includes its copy of the Go runtime and the Go standard library. Bhargavan et al. [4] impose substantial restrictions on the API of verified code, e.g., disallowing state preservation between API calls. Codebases do not normally satisfy these restrictions, including all our case studies. E.g., they use a session key for sending a transport message in one API call that was established during the handshake, i.e., a previous API call.

Existing implementations. Dupressoir et al. [13] embed a *trace* storing relevant protocol operations as an auxiliary data structure for proof purposes into C code implementing a security protocol. This auxiliary data structure is removed before compilation and does not incur any runtime overhead while enabling reasoning about weak secrecy and non-injective agreement. Arqunt et al. [15] generalize this approach to separation logic, making it applicable to a wide range of programming languages and supporting stronger security properties such as forward secrecy and injective agreement. However, both approaches require a sufficiently strong invariant over this trace to prove security properties. To avoid such a trace invariant, Arqunt et al. [14] prove security properties on the level of an abstract model using Tamarin’s proof automation and prove that an implementation refines the abstract model. All three approaches require verifying the entire codebase using an auto-active verifier (which we do not). We build on the latter approach and, to the best of our knowledge, are the first to relax this requirement to verifying just the *CORE* and reason about the *APPLICATION* using lightweight static analyses.

Dynamic verification. Several approaches employ dynamic checks at runtime to allow for partially verified codebases. Agten et al. [46] target single-threaded C code and generate runtime checks at the boundary between verified and unverified code to test that the verified code’s specification holds. To detect violations of properties expressed in separation logic such as ownership (via permissions) and aliasing, this approach tracks the heap locations accessed by the verified codebase at runtime and computes cryptographic hashes thereof. It remains unclear whether these checks only at the boundary remain sufficient when targeting concurrent codebases or whether the runtime overhead increases further.

To avoid tracking heap locations at runtime, Ho et al. [47] copy all heap data at this boundary to rule out aliasing.

Gradual verification (e.g., [48], [49]) combines auto-active verification with dynamic checks but aims at helping the proof developer by allowing incomplete specifications. I.e., gradual verification enables incremental verification where each function’s specification is extended over time to eventually obtain a fully specified and verified codebase. However, as long as a codebase is not fully specified and verified, gradual verification requires tracking heap locations at runtime, which results in noticeable runtime overhead.

SCIO* [50] is an F* transpiler that injects dynamic checks not only at the boundary between verified and unverified code but also at call sites of I/O operations. While they can enforce access policies for I/O operations, it remains unclear how this approach extends to cryptographic message payloads. To be applicable in our context, we would need to dynamically check whether a message sent by our *APPLICATION* is indeed protocol-irrelevant and, thus, does not contain any secrets from the *CORE*—not even in encrypted form. Like our work, SecRef* [51] considers the problem of verifying only a subset of a codebase due to the otherwise prohibitive proof effort. While they also allow pre- and postconditions at the boundary between verified and unverified code, they rely on dynamic checks to enforce these conditions for heap locations accessible by unverified code. For a verified component like our *CORE*, this means that they check the entire invariant at runtime for each API call (which we do not), as they treat the unverified code as potentially modifying a *CORE* instance’s entire state. By targeting a single-threaded language (F*), SecRef* does not have to consider concurrent memory accesses (which we do).

By contrast, *DIODON* performs only extremely lightweight dynamic checks enforcing non-nilness and correct ordering of API calls, and checks all other constraints *statically* to avoid runtime overhead while simultaneously requiring minimal code changes.

7. Conclusions

We present *DIODON*, a novel methodology to scale verification of security protocol implementations to large existing codebases by symbiotically combining powerful auto-active verification of a relatively small part of the codebase with static analyses that scale to the entire codebase. Since *DIODON* is not inherently limited to Go, future work could apply it to codebases written in, e.g., Rust and C. Adapting *DIODON* to Rust would remove several static analyses due to the strong type system, and C has a variety of static analyses and auto-active verifiers that can be used. Orthogonally, extending *DIODON* with further static analyses, such as for nilness, would allow us to pass more guarantees from the *APPLICATION* to the *CORE*. We hope our work spurs both practical and theoretical understanding of how to soundly combine proof systems of different expressive power.

References

- [1] B. Schmidt, S. Meier, C. Cremers, and D. A. Basin, “Automated analysis of diffie-hellman protocols and advanced security properties,” in *CSF*. IEEE Computer Society, 2012, pp. 78–94.
- [2] S. Meier, B. Schmidt, C. Cremers, and D. A. Basin, “The TAMARIN prover for the symbolic analysis of security protocols,” in *CAV*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 696–701.
- [3] B. Blanchet, “An efficient cryptographic protocol verifier based on prolog rules,” in *CSFW*. IEEE Computer Society, 2001, pp. 82–96.
- [4] K. Bhargavan, B. Blanchet, and N. Kobeissi, “Verified models and reference implementations for the TLS 1.3 standard candidate,” in *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2017, pp. 483–502.
- [5] D. A. Basin, R. Sasse, and J. Toro-Pozo, “The EMV standard: Break, fix, verify,” in *SP*. IEEE, 2021, pp. 1766–1781.
- [6] N. Kobeissi, K. Bhargavan, and B. Blanchet, “Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach,” in *EuroS&P*. IEEE, 2017, pp. 435–450.
- [7] D. A. Basin, J. Dreier, L. Hirschi, S. Radomirovic, R. Sasse, and V. Stettler, “A formal analysis of 5g authentication,” in *CCS*. ACM, 2018, pp. 1383–1396.
- [8] C. Cremers and M. Dehnel-Wild, “Component-based formal analysis of 5g-aka: Channel assumptions and session confusion,” in *NDSS*. The Internet Society, 2019.
- [9] CVE, “CVE-2014-0160,” 2013. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2014-0160>
- [10] —, “CVE-2021-40823,” 2021. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2021-40823>
- [11] —, “CVE-2022-22805,” 2022. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2022-22805>
- [12] —, “CVE-2022-22806,” 2022. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2022-22806>
- [13] F. Dupressoir, A. D. Gordon, J. Jürjens, and D. A. Naumann, “Guiding a general-purpose C verifier to prove cryptographic protocols,” in *CSF*. IEEE Computer Society, 2011, pp. 3–17.
- [14] L. Arquint, F. A. Wolf, J. Lallemand, R. Sasse, C. Sprenger, S. N. Wiesner, D. A. Basin, and P. Müller, “Sound verification of security protocols: From design to interoperable implementations,” in *SP*. IEEE, 2023, pp. 1077–1093.
- [15] L. Arquint, M. Schwerhoff, V. Mehta, and P. Müller, “A generic methodology for the modular verification of security protocol implementations,” in *CCS*. ACM, 2023, pp. 1377–1391.
- [16] D. Dolev and A. C. Yao, “On the security of public key protocols,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–207, 1983.
- [17] CVE, “CVE-2024-47083,” 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2024-47083>
- [18] —, “CVE-2023-6746,” 2023. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2023-6746>
- [19] K. R. M. Leino and M. Moskal, “Usable auto-active verification,” in *Usable Verification Workshop*, 2010.
- [20] Amazon Web Services, Inc., “Working with SSM Agent,” 2023. [Online]. Available: <https://docs.aws.amazon.com/systems-manager/latest/userguide/ssm-agent.html>
- [21] F. A. Wolf, L. Arquint, M. Clochard, W. Oortwijn, J. C. Pereira, and P. Müller, “Gobra: Modular specification and verification of go programs,” in *CAV (I)*, ser. Lecture Notes in Computer Science, vol. 12759. Springer, 2021, pp. 367–379.
- [22] AWS Labs, “Argot,” 2024. [Online]. Available: <https://github.com/aws-labs/ar-go-tools>
- [23] J. C. Reynolds, “Separation logic: A logic for shared mutable data structures,” in *LICS*. IEEE Computer Society, 2002, pp. 55–74.
- [24] D. A. Basin, X. Hofmeier, R. Sasse, and J. Toro-Pozo, “Getting chip card payments right,” in *FM (I)*, ser. Lecture Notes in Computer Science, vol. 14933. Springer, 2024, pp. 29–51.
- [25] G. Girol, L. Hirschi, R. Sasse, D. Jackson, C. Cremers, and D. A. Basin, “A spectral analysis of noise: A comprehensive, automated, formal analysis of diffie-hellman protocols,” in *USENIX Security Symposium*. USENIX Association, 2020, pp. 1857–1874.
- [26] J. Boyland, “Checking interference with fractional permissions,” in *SAS*, ser. Lecture Notes in Computer Science, vol. 2694. Springer, 2003, pp. 55–72.
- [27] M. J. Parkinson and G. M. Bierman, “Separation Logic and abstraction,” in *POPL*. ACM, 2005, pp. 247–258.
- [28] B. Jacobs, J. Smans, P. Philippaerts, F. Vogels, W. Penninckx, and F. Piessens, “VeriFast: A powerful, sound, predictable, fast verifier for C and Java,” in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 6617. Springer, 2011, pp. 41–55.
- [29] Q. Cao, L. Beringer, S. Gruetter, J. Dodds, and A. W. Appel, “VST-Floyd: A separation logic tool to verify correctness of C programs,” *Journal of Automated Reasoning*, vol. 61, no. 1, pp. 367–422, Jun 2018.
- [30] M. Eilers and P. Müller, “Nagini: A static verifier for python,” in *CAV (I)*, ser. Lecture Notes in Computer Science, vol. 10981. Springer, 2018, pp. 596–603.
- [31] V. Astrauskas, P. Müller, F. Poli, and A. J. Summers, “Leveraging Rust types for modular specification and verification,” *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, pp. 147:1–147:30, 2019.
- [32] W. Penninckx, B. Jacobs, and F. Piessens, “Sound, modular and compositional verification of the input/output behavior of programs,” in *ESOP*, ser. Lecture Notes in Computer Science, vol. 9032. Springer, 2015, pp. 158–182.
- [33] Google, “Capslock,” 2024. [Online]. Available: <https://github.com/google/capslock>
- [34] J. Protzenko, B. Parno, A. Fromherz, C. Hawblitzel, M. Polubelova, K. Bhargavan, B. Beurdouche, J. Choi, A. Delignat-Lavaud, C. Fournet, N. Kulatova, T. Ramanananandro, A. Rastogi, N. Swamy, C. M. Wintersteiger, and S. Z. Béguelin, “EverCrypt: A fast, verified, cross-platform cryptographic provider,” in *S&P*. IEEE, 2020, pp. 983–1002.
- [35] Amazon Web Services, Inc., “AWS Key Management Service,” 2024. [Online]. Available: <https://aws.amazon.com/kms/>
- [36] M. Barbosa, G. Barthe, K. Bhargavan, B. Blanchet, C. Cremers, K. Liao, and B. Parno, “SoK: Computer-aided cryptography,” in *S&P*. IEEE, 2021, pp. 777–795.
- [37] M. Avale, A. Pironti, and R. Sisto, “Formal verification of security protocol implementations: a survey,” *Formal Aspects Comput.*, vol. 26, no. 1, pp. 99–123, 2014.
- [38] B. Blanchet, “Security protocol verification: Symbolic and computational models,” in *POST*, ser. Lecture Notes in Computer Science, vol. 7215. Springer, 2012, pp. 3–29.
- [39] D. Pozza, R. Sisto, and L. Durante, “Spi2Java: Automatic cryptographic protocol Java code generation from spi calculus,” in *AINA*. IEEE Computer Society, 2004, pp. 400–405.
- [40] D. Cadé and B. Blanchet, “From computationally-proved protocol specifications to implementations,” in *ARES*. IEEE Computer Society, 2012, pp. 65–74.
- [41] K. Bhargavan, A. Bichhawat, Q. H. Do, P. Hosseini, R. Küsters, G. Schmitz, and T. Würtele, “DY*: A modular symbolic verification framework for executable cryptographic protocol code,” in *EuroS&P*. IEEE, 2021, pp. 523–542.

- [42] J. Gancher, S. Gibson, P. Singh, S. Dharanikota, and B. Parno, “OWL: Compositional verification of security protocols via an information-flow type system,” in *SP*. IEEE, 2023, pp. 1130–1147.
- [43] K. Bhargavan, C. Fournet, A. D. Gordon, and S. Tse, “Verified interoperable implementations of security protocols,” *ACM Trans. Program. Lang. Syst.*, vol. 31, no. 1, pp. 5:1–5:61, 2008.
- [44] N. O’Shea, “Using Elyjah to analyse Java implementations of cryptographic protocols,” in *FCS-ARSPA-WITS-2008*, 2008.
- [45] M. Aizatulin, A. D. Gordon, and J. Jürjens, “Computational verification of C protocol implementations by symbolic execution,” in *CCS*. ACM, 2012, pp. 712–723.
- [46] P. Agten, B. Jacobs, and F. Piessens, “Sound modular verification of C code executing in an unverified context,” in *POPL*. ACM, 2015, pp. 581–594.
- [47] S. Ho, J. Protzenko, A. Bichhawat, and K. Bhargavan, “Noise*: A library of verified high-performance secure channel protocol implementations,” in *SP*. IEEE, 2022, pp. 107–124.
- [48] J. Bader, J. Aldrich, and É. Tanter, “Gradual program verification,” in *VMCAI*, ser. Lecture Notes in Computer Science, vol. 10747. Springer, 2018, pp. 25–46.
- [49] J. Wise, J. Bader, C. Wong, J. Aldrich, É. Tanter, and J. Sunshine, “Gradual verification of recursive heap data structures,” *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, pp. 228:1–228:28, 2020.
- [50] C. Andrici, Ş. Ciobăcă, C. Hritcu, G. Martínez, E. Rivas, É. Tanter, and T. Winterhalter, “Securing verified IO programs against unverified code in F*,” *Proc. ACM Program. Lang.*, vol. 8, no. POPL, pp. 2226–2259, 2024.
- [51] C. Andrici, D. Ahman, C. Hritcu, R. Icleanu, G. Martínez, E. Rivas, and T. Winterhalter, “Secref*: Securely sharing mutable references between verified and unverified code in F,” *CoRR*, vol. abs/2503.00404, 2025.
- [52] L. Arqunt, F. A. Wolf, J. Lallemand, R. Sasse, C. Sprenger, S. N. Wiesner, D. A. Basin, and P. Müller, “Sound verification of security protocols: From design to interoperable implementations (extended version),” *CoRR*, vol. abs/2212.04171, 2022.
- [53] C. Sprenger, T. Klenze, M. Eilers, F. A. Wolf, P. Müller, M. Clochard, and D. A. Basin, “Igloo: soundly linking compositional refinement and separation logic for distributed system verification,” *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, pp. 152:1–152:31, 2020.
- [54] V. Vafeiadis, “Concurrent separation logic and operational semantics,” in *MFPS*, ser. Electronic Notes in Theoretical Computer Science, vol. 276. Elsevier, 2011, pp. 335–351.

Appendix A.

Secure Shell Session Protocol

Fig. 10 shows the protocol for establishing interactive shell sessions between an SSM Agent (A) and an AWS customer (B). The protocol includes two additional roles namely KMS (S) and an optional, trusted monitor (M) that is allowed to inspect the established shell sessions, e.g., for compliance reasons.

Since A and B do not personally possess their secret keys for creating signatures, we explicitly model the presence of and the interactions with KMS that remotely creates and verifies signatures. We model these interactions as happening on a secure channel, indicated by $\Rightarrow$, because each role instance of A and B establishes a TLS connection to KMS.

On a high-level, this protocol performs a signed elliptic-curve Diffie-Hellman key exchange establishing two symmetric keys $kdf1(g^{x*y})$ and $kdf2(g^{x*y})$. These keys are used

in the transport phase, i.e., $M15$ and $M16$, to symmetrically encrypt (*senc*) payloads for sending in a particular direction. In Tamarin, we model the transport phase as a non-deterministic loop that allows each role A and B to send and receive an unbounded amount of transport messages and interleave them arbitrarily.

More specifically, the protocol proceeds as follows. A first generates an elliptic-curve public-private keypair, which we model in Tamarin as generating a random number x and computing the public component via modular exponentiation denoted by g^x . Then, A uses $M1$ to instruct KMS to use a particular signing key belonging to A, identified by Id_{skA} , to sign the triple $\langle g^x, Id_M, Id_B \rangle$. This triple includes the monitor’s and B’s identity to prevent person-in-the-middle (PITM) attacks. If the requested signing key actually belongs to A, KMS creates and sends the signature in $M2$ back to A. This allows A to send a session request ($M3$) to B, which includes g^x , the signature, and signing key’s and monitor’s identities.

After receiving a session request, B first verifies the received signature via KMS. For this purpose, B sends the signature itself and the components over which the signature is computed in the verify request ($M4$) to KMS. If the signature is valid, KMS replies with a verification response ($M5$). Otherwise, KMS aborts the protocol, which we model as not sending any response. Afterwards, B generates its elliptic-curve public-private keypair (g^y, y) and uses KMS to sign g^y and A’s identity. B then sends a session response ($M8$) to A that contains B’s public curve point, the signature, the identity of B’s signing key, and a hash of the shared secret $h(g^{x*y})$. The latter allows A to detect early on if A and B computed different shared secrets, e.g., due to an attempted replay attack.

After receiving a session response, A computes the shared secret and checks that it derives the same shared secret’s hash value. Additionally, A verifies the received signature using KMS and derives the two symmetric session keys from the shared secret by applying two different key derivation functions $kdf1$ and $kdf2$. To enable a trusted monitor to audit the shell session, A computes c_{ss} by asymmetrically encrypting the two session keys using the monitor’s public key pk_M . Next, A requests a signature from KMS for c_{ss} and B’s identity to bind these identities to the session keys. The handshake completes by sending the encrypted session keys to the monitor ($M13$) and confirming the session keys to B ($M14$). The latter message includes some version information, which we model as an adversary-chosen payload z .

Message $M13$ enables M, a trusted third party, to monitor the transmitted shell commands should this be necessary for regulatory reasons (otherwise sending message $M3$ can simply be skipped). For this purpose, the SSM Agent A sends the asymmetrically encrypted session keys to the monitor M such that M can obtain the session keys and, thus, decrypt and audit the transport messages. Note that the monitor does not need to be online during the handshake. Instead, an untrusted log server could store message $M3$ until M becomes online and fetches this message from the log server.

$M1. A \Rightarrow S : \langle \text{SignReq}, Id_{skA}, g^x, Id_M, Id_B \rangle$
 $M2. S \Rightarrow A : \langle \text{SignResp}, sig_x \rangle$
 $M3. A \rightarrow B : \langle \text{SessReq}, g^x, sig_x, Id_{skA}, Id_M \rangle$
 $M4. B \Rightarrow S : \langle \text{VerReq}, Id_A, Id_{skA}, g^x, Id_M, Id_B, sig_x \rangle$
 $M5. S \Rightarrow B : \langle \text{VerResp} \rangle$
 $M6. B \Rightarrow S : \langle \text{SignReq}, Id_{skB}, g^y, Id_A \rangle$
 $M7. S \Rightarrow B : \langle \text{SignResp}, sig_y \rangle$
 $M8. B \rightarrow A : \langle \text{SessResp}, g^y, sig_y, Id_{skB}, h(g^{x*y}) \rangle$
 $M9. A \Rightarrow S : \langle \text{VerReq}, Id_B, Id_{skB}, g^y, Id_A, sig_y \rangle$
 $M10. S \Rightarrow A : \langle \text{VerResp} \rangle$
 $M11. A \Rightarrow S : \langle \text{SignReq}, Id_{skA}, c_{ss}, Id_B \rangle$
 $M12. S \Rightarrow A : \langle \text{SignResp}, sig_{ss} \rangle$
 $M13. A \rightarrow M : \langle \text{SSKey}, c_{ss}, sig_{ss}, Id_A, Id_{skA}, Id_B \rangle$
 $M14. A \rightarrow B : \langle \text{HSDone}, c_{ss}, senc(\langle \text{HSPay}, z \rangle, kdf1(g^{x*y})) \rangle$
 $M15. A \rightarrow B : \langle \text{Msg}, senc(z, kdf1(g^{x*y})) \rangle$
 $M16. B \rightarrow A : \langle \text{Msg}, senc(z, kdf2(g^{x*y})) \rangle$

where

$$\begin{aligned}
sig_x &\triangleq \text{sign}(\langle g^x, Id_M, Id_B \rangle, sk_A) \\
sig_y &\triangleq \text{sign}(\langle g^y, Id_A \rangle, sk_B) \\
c_{ss} &\triangleq \text{aenc}(\langle kdf1(g^{x*y}), kdf2(g^{x*y}) \rangle, pk_M) \\
sig_{ss} &\triangleq \text{sign}(\langle c_{ss}, Id_B \rangle, sk_A)
\end{aligned}$$

Figure 10. Signed Diffie-Hellman key exchange for deriving the symmetric keys $kdf1(g^{x*y})$ and $kdf2(g^{x*y})$ that are used during the transport phase, i.e., in messages $M15$ and $M16$. We use $\rightarrow$ and $\Rightarrow$ to denote communication via the untrusted network and a secure channel, respectively.

Appendix B. Soundness Proof Sketch

To prove **DION** sound, we reason separately about allowing protocol-independent I/O operations in a codebase and combining auto-active verification with static analyses.

In App. B.1, we prove that a codebase c containing protocol-dependent *and* protocol-independent I/O operations refines a given Tamarin model if the I/O specification ϕ , corresponding to a protocol role in this Tamarin model, permits all protocol-dependent I/O operations in the codebase. For this part of the soundness proof, we assume that the *entire* codebase c satisfies the Hoare triple $[\phi] \ c \ [\text{true}]$, where protocol-independent I/O operations do not consume an I/O permission and, thus, can be performed at arbitrary points within c and independently of the I/O specification ϕ . Such a Hoare triple could be obtained by auto-actively verifying the *entire* codebase c , which **DION** does not require.

In App. B.2, we show that we constructively obtain the Hoare triple $[\phi] \ c \ [\text{true}]$ for an entire codebase c using **DION** by auto-actively verifying only parts thereof, namely the **CORE**, and executing our static analyses on c .

By combining both results, we obtain that applying **DION** to a codebase c proves that c refines a Tamarin model, despite the presence of protocol-independent I/O operations and auto-actively verifying only the **CORE**, as long as we discharge the side conditions using our static analyses.

B.1. I/O Independence

We show that we can soundly allow protocol-independent I/O operations in a codebase by treating these

I/O operations as a refinement of our attacker model. For this purpose, we extend Arqunt et al.'s [52] soundness proof to accommodate such I/O operations and we adopt their notation for legibility. More specifically, we add these I/O operations to the concrete system and show that this concrete system refines an abstract system that is composed of only protocol roles and our attacker, which corresponds to a protocol's Tamarin model.

Since we permit a codebase c to perform independent I/O operations in addition to I/O operations permitted by an I/O specification ϕ , we adapt the verifier assumption [52, Assumption 1] to account for both types of I/O operations.

Assumption 1 (Verifier assumption).

$$\vdash_{\alpha} [\phi] \ c \ [\text{true}] \wedge \mathbb{T}(c, s) = \text{true} \implies \alpha(\mathcal{C}) \preceq \phi \parallel \varphi.$$

I.e., we assume that successfully verifying a program c against an I/O specification ϕ and successfully executing the taint analysis $\mathbb{T}$ with some configuration s specifying sources and sinks of tainted data implies that the program's traces abstracted under a relabeling function α are included in the parallel composition of the I/O specification's traces ϕ and the traces of performing independent I/O operations φ .

We assume that the program's traces are described by the labeled transition system (LTS) semantics $\mathcal{C}$, which is provided by the operational semantics of the programming language in which c is implemented². α abstracts the program's traces, e.g., referring to specific function names, to traces of operations that match the naming as used in ϕ and φ .

φ represents the set of traces resulting from generating fresh nonces and using received payloads as well as public constants to construct and send payloads as will be made explicit in Def. 2.

Note that Asm. 1 expresses besides the trace inclusion itself that the states of ϕ and φ are independent such that their parallel composition is possible. We obtain this independence by successfully executing our taint analysis. In particular, our taint analysis establishes that protocol-independent I/O operations do not operate on tainted data. We configure the taint analysis such that long-term and short-term secrets known by a protocol role but not the attacker are a source of taint. Therefore, we show that every protocol-independent I/O operation uses data that is independent of these secrets. Hence, these I/O operations and all steps necessary to compute their data are part of φ and independent from ϕ .

The other direction, namely that the I/O specification ϕ is independent of from φ , holds by construction of ϕ . Since we generate ϕ by a series of transformations from a protocol role's abstract model and use syntactically distinct elements to represent this protocol role's state and express the transitions that form φ , as we shall see next, φ cannot influence ϕ .

For a set of function symbols Σ operating over terms, MD denotes the set of transition rules that the attacker can

2. We leave the programming language intentionally unspecified to keep our soundness result general.

apply. $K(x)$ represents the fact that the attacker knows the term x and the fact symbols out and in represent that a protocol participant sent and might receive a particular term, respectively. Therefore, MD captures all operations available to the attacker, namely receiving a sent term, sending a term, adding a public constant to its knowledge, generating a fresh nonce, and applying a function $f \in \Sigma$.

Definition 1 (Attacker message deduction rules). *As defined in [52, Sec. E.1 Def. 9], MD_Σ denotes the set of message deduction rules representing our DY attacker for Σ :*

$$\begin{aligned} & [\text{out}(x)] \xrightarrow{\emptyset} [K(x)] \\ & [K(x)] \xrightarrow{[K(x)]} [\text{in}(x)] \\ & \emptyset \xrightarrow{\emptyset} [K(x \in \text{pub})] \\ & [\text{Fr}(x \in \text{fresh})] \xrightarrow{\emptyset} [K(x)] \\ & [K(x_1), \dots, K(x_k)] \xrightarrow{\emptyset} [K(f(x_1, \dots, x_k))] \\ & \quad \text{for } f \in \Sigma \text{ with arity } k \end{aligned}$$

Similarly, we define MD_Σ^{ind} , which consists of the transition rules a protocol-independent component can execute. In contrast to MD , MD_Σ^{ind} operates on syntactically different, reserved fact symbols. Avoiding these name clashes simplifies defining a simulation relation for proving Lemma 1.

While ind represents knowledge of a particular term, out_{ind} and in_{ind} represent a term sent to and received from the network, respectively. ind , out_{ind} , and in_{ind} are in the same class of fact symbols as their analogous counterparts K , out , and in , respectively. I.e., K and ind are persistent fact symbols Σ_{per} capturing the property that knowledge is monotonically increasing. This means that applying a transition rule does not consume such facts and, thus, their multiplicity in the multiset comprising the state is irrelevant. In contrast, out , in , out_{ind} , and in_{ind} are in the class of linear fact symbols Σ_{lin} , meaning that applying a transition rule that states such a fact in its premise will remove this fact from the state while such a fact occurring in the rule's conclusion adds it to the state. Additionally, out_{ind} and in_{ind} are in the class of output and input fact symbols Σ_{out} and Σ_{in} , respectively, as suggested by their intuitive semantics.

Definition 2 (Protocol-independent message deduction rules).

$$\begin{aligned} & [\text{ind}(x)] \xrightarrow{\emptyset} [\text{out}_{\text{ind}}(x)] \\ & [\text{in}_{\text{ind}}(x)] \xrightarrow{\emptyset} [\text{ind}(x)] \\ & \emptyset \xrightarrow{\emptyset} [\text{ind}(x \in \text{pub})] \\ & [\text{Fr}(x \in \text{fresh})] \xrightarrow{\emptyset} [\text{ind}(x)] \\ & [\text{ind}(x_1), \dots, \text{ind}(x_k)] \xrightarrow{\emptyset} [\text{ind}(f(x_1, \dots, x_k))] \\ & \quad \text{for } f \in \Sigma \text{ with arity } k \end{aligned}$$

where ind , out_{ind} , and in_{ind} are reserved fact symbols and $\text{ind} \in \Sigma_{\text{per}}$, $\text{out}_{\text{ind}} \in \Sigma_{\text{out}} \cap \Sigma_{\text{lin}}$, and $\text{in}_{\text{ind}} \in \Sigma_{\text{in}} \cap \Sigma_{\text{lin}}$.

Although we present MD_Σ^{ind} on the same abstraction level as the attacker deduction rules MD_Σ to make them

more legible, these deduction rules are *not* part of the MRS $\mathcal{R}$. Instead, we transform these rules according to [52] and make them part of the component system as described next.

We introduce buffered versions for the out_{ind} and in_{ind} facts and split the rules in MD_Σ^{ind} involving I/O into two separate rules each, which we synchronize using transition labels. This split allows us to assign half of the rules to the component executing protocol-independent operations $\mathcal{R}_{\text{ind}}(\text{rid})$ and assign the remaining rules $\mathcal{R}_{\text{io}}^+$ to the environment forming $\mathcal{R}_{\text{env}}^{e+}$. We use χ^+ to synchronize the execution of these now separated rules.

Definition 3 ($\mathcal{R}_{\text{ind}}(\text{rid})$). $\mathcal{R}_{\text{ind}}(\text{rid})$ consists of the following multiset transition rules.

$$\begin{aligned} & [\text{ind}(\text{rid}, x)] \xrightarrow{[\lambda_{\text{out}_{\text{ind}}}^s(\text{rid}, x)]} \emptyset \\ & \emptyset \xrightarrow{[\lambda_{\text{in}_{\text{ind}}}^s(\text{rid}, x)]} [\text{ind}(\text{rid}, x)] \\ & \emptyset \xrightarrow{\emptyset} [\text{ind}(\text{rid}, x \in \text{pub})] \\ & \emptyset \xrightarrow{[\lambda_{\text{Fr}_{\text{ind}}}^s(\text{rid}, x)]} [\text{ind}(\text{rid}, x)] \\ & \left[\begin{array}{c} \text{ind}(\text{rid}, x_1), \\ \dots, \\ \text{ind}(\text{rid}, x_k) \end{array} \right] \xrightarrow{\emptyset} [\text{ind}(\text{rid}, f(x_1, \dots, x_k))] \\ & \quad \text{for } f \in \Sigma \text{ with arity } k \end{aligned}$$

Definition 4 ($\mathcal{R}_{\text{env}}^{e+}$). $\mathcal{R}_{\text{env}}^{e+} = \mathcal{R}_{\text{env}}^e \uplus \mathcal{R}_{\text{io}}^+$ where $\mathcal{R}_{\text{env}}^e$ is defined as in [52, Sec. 3.2.2 (6)] and $\mathcal{R}_{\text{io}}^+$ consists of the following multiset transition rules.

$$\begin{aligned} & \emptyset \xrightarrow{[\lambda_{\text{out}_{\text{ind}}}^e(\text{rid}, x)]} [\text{out}_{\text{ind}}(x)] \\ & [\text{in}_{\text{ind}}(x)] \xrightarrow{[\lambda_{\text{in}_{\text{ind}}}^e(\text{rid}, x)]} \emptyset \\ & [\text{Fr}(x \in \text{fresh})] \xrightarrow{[\lambda_{\text{Fr}_{\text{ind}}}^e(\text{rid}, x)]} \emptyset \end{aligned}$$

Definition 5 (χ^+). We define the partial synchronization function $\chi^+ : (\bigcup_{i, \text{rid}} (\mathcal{R}_{\text{role}}^i(\text{rid}) \cup \mathcal{R}_{\text{ind}}(\text{rid}))) \times \mathcal{R}_{\text{env}}^{e+} \rightarrow \mathcal{E}$ that synchronizes events of the two systems $\mathcal{R}_{\text{role}}^i(\text{rid}) \parallel \mathcal{R}_{\text{ind}}(\text{rid})$ and $\mathcal{R}_{\text{env}}^{e+}$, i.e.,

$$\chi^+(e, e') = \begin{cases} \epsilon & \text{if } e = F^s(\text{rid}, x) \text{ and } e' = F^e(\text{rid}, x) \\ \chi(e, e') & \text{if } e \neq F^s(\text{rid}, x) \text{ and } e' \neq F^e(\text{rid}, x) \\ \text{undef.} & \text{otherwise} \end{cases}$$

where $F \in \{\lambda_{\text{out}_{\text{ind}}}, \lambda_{\text{in}_{\text{ind}}}, \lambda_{\text{Fr}_{\text{ind}}}\}$ and the partial function χ [52, Sec. E.5] synchronizes labels occurring in $\mathcal{R}_{\text{role}}^i$ and $\mathcal{R}_{\text{env}}^e$.

Lemma 1.

$$\begin{aligned} & \left(\mathcal{R}_{\text{role}}^i(\text{rid}) \parallel \mathcal{R}_{\text{ind}}(\text{rid}) \right) \parallel_{\chi^+} \mathcal{R}_{\text{env}}^{e+} \\ & \preceq \left(\mathcal{R}_{\text{role}}^i(\text{rid}) \right) \parallel_{\chi} \mathcal{R}_{\text{env}}^e \end{aligned}$$

Given $\mathcal{R}_{\text{ind}}(\text{rid})$ and $\mathcal{R}_{\text{env}}^{e+}$, Lemma 1 states that we can treat the system (on the first line) consisting of possibly

unboundedly-many instances of components executing a protocol role and executing protocol-independent operations as a refinement of the system on the second line that does not have components executing protocol-independent operations and uses an environment without the rules in $\mathcal{R}_{io}^+$.

The following proof proceeds by setting up a simulation relation that merges the states of components executing protocol-independent operations with the environment and renames certain fact symbols. Using this simulation relation, we show that each transition in the concrete system can be simulated by the abstract system. While this simulation is straightforward for transitions executed by components that are present in the concrete and abstract system, the concrete transitions corresponding to protocol-independent operations are more insightful as we show that the abstract environment, namely our DY attacker model, can simulate those transitions.

Proof. We denote $\mathcal{E} = \left(\parallel_{i,rid} \mathcal{R}_{role}^i(rid) \right) \parallel_{\chi} \mathcal{R}_{env}^e$ and $\mathcal{E}' = \left(\parallel_{i,rid} (\mathcal{R}_{role}^i(rid) \parallel \mathcal{R}_{ind}(rid)) \right) \parallel_{\chi^+} \mathcal{R}_{env}^{e+}$ the abstract and concrete systems, respectively, and prove this lemma by establishing a refinement with a simulation relation $\mathcal{R}$ between states of the abstract system $\mathcal{E}$ and states of the concrete system $\mathcal{E}'$. Accordingly, we use $\rightarrow_{\mathcal{E}}$ and $\rightarrow_{\mathcal{E}'}$ to denote a transition step in the abstract system $\mathcal{E}$ and concrete system $\mathcal{E}'$, respectively. Additionally, we use $\rightarrow_{\mathcal{R}_{role}^i(rid)}$ and $\rightarrow_{\mathcal{R}_{env}^e}$ for transitions performed by the individual components in the abstract system and, similarly, $\rightarrow_{\mathcal{R}_{role}^i(rid)}$, $\rightarrow_{\mathcal{R}_{ind}(rid)}$ and $\rightarrow_{\mathcal{R}_{env}^{e+}}$ for the concrete system's components. The abstract states are of the shape $((s_{i,rid})_{1 \leq i \leq n, \text{ for each } rid, s_e})$. We use primed variables for referring to concrete states, which are of the shape $((s'_{i,rid}, s'_{ind,i,rid})_{1 \leq i \leq n, \text{ for each } rid, s'_e})$, i.e., they are composed of two multisets of facts for each i, rid , and one for the environment. Each multiset $s'_{i,rid}$ corresponds to the state of instance rid executing the protocol role i , while $s'_{ind,i,rid}$ corresponds to the state of the component executing protocol-independent operations, which is conceptually co-located with $s'_{i,rid}$ but guaranteed by our taint analysis to operate on distinct state.

We use the refinement relation $(s, s') \in \mathcal{R}$ iff

$$s = ((s'_{i,rid})_{1 \leq i \leq n, \text{ for each } rid, r((\cup_{i,rid}^m s'_{ind,i,rid}) \cup^m s'_e)}),$$

where $s' = ((s'_{i,rid}, s'_{ind,i,rid})_{1 \leq i \leq n, \text{ for each } rid, s'_e})$ and r is the identity function except for the cases specified below. We lift r to operate on multiset of facts. This lifted version removes duplicate K facts because K is a persistent fact symbol.

$$\left. \begin{aligned} r(\text{ind}(rid, x)) &= K(x) \\ r(\text{out}_{ind}(x)) &= K(x) \\ r(\text{in}_{ind}(x)) &= K(x) \end{aligned} \right\} \text{ for all } rid, x.$$

I.e., to derive s_e from s' , we, first, combine all facts in the states of protocol-independent components $s'_{ind,i,rid}$ with s'_e by applying multiset union $\cup^m$ and, second, rename and deduplicate these facts according to the renaming function r .

It is clear that the initial states are related, i.e., $(s, s') \in \mathcal{R}$ with $s = ((\emptyset, \dots, \emptyset), \emptyset)$ and $s' = ((\emptyset, \emptyset), \dots, (\emptyset, \emptyset), \emptyset)$. We now show that for all states $(s_1, s'_1) \in \mathcal{R}$ and for all concrete transition steps $s'_1 \xrightarrow{e}_{\mathcal{E}'} s'_2$ there exists an abstract transition $s_1 \xrightarrow{e}_{\mathcal{E}} s_2$ such that $(s_2, s'_2) \in \mathcal{R}$. We use the following naming convention to refer to individual multisets within the abstract and concrete states, respectively, for $j \in \{1, 2\}$:

$$\begin{aligned} s_j &= ((s_{j,i,rid})_{1 \leq i \leq n, \text{ for each } rid, s_{j,e}}) \\ s'_j &= ((s'_{j,i,rid}, s'_{j,ind,i,rid})_{1 \leq i \leq n, \text{ for each } rid, s'_{j,e}}) \end{aligned}$$

Based on the definition of the parallel and synchronizing composition, $\parallel$ and $\parallel_{\chi^+}$, resp., we distinguish the following two cases for the transition step $s'_1 \xrightarrow{e}_{\mathcal{E}'} s'_2$:

- $e = \chi^+(F^s(rid, x), F^e(rid, x))$ for $F \in \{\lambda_{\text{out}_{ind}}, \lambda_{\text{in}_{ind}}, \lambda_{F_{ind}}\}$:
Since $s'_1 \xrightarrow{e}_{\mathcal{E}'} s'_2$, we have:

$$\begin{aligned} s'_{1,ind,i,rid} &\xrightarrow{F^s(rid,x)}_{\mathcal{R}_{ind}(rid)} s'_{2,ind,i,rid} \\ s'_{1,e} &\xrightarrow{F^e(rid,x)}_{\mathcal{R}_{env}^{e+}} s'_{2,e} \end{aligned}$$

and all other component states remain unchanged, i.e.,

$$\begin{aligned} s'_{2,i,rid} &= s'_{1,i,rid} \\ s'_{2,j,rid'} &= s'_{1,j,rid'} \\ s'_{2,ind,j,rid'} &= s'_{1,ind,j,rid'} \end{aligned}$$

for all $(j, rid') \neq (i, rid)$. We now need to distinguish the cases where $F = \lambda_{\text{out}_{ind}}$, $F = \lambda_{\text{in}_{ind}}$, and $F = \lambda_{F_{ind}}$.

- $F = \lambda_{\text{out}_{ind}}$: By definition of the transition rule F^s , we have $\text{ind}(rid, x) \in s'_{1,ind,i,rid}$ and $s'_{2,ind,i,rid} = s'_{1,ind,i,rid} \setminus^m \{\text{ind}(rid, x)\}$. Similarly by definition of F^e , we have $s'_{2,e} = s'_{1,e} \cup^m \{\text{out}_{ind}(x)\}$. By definition of χ^+ , we simulate this transition in $\mathcal{E}$ by performing the empty transition ϵ , i.e., $s_1 \xrightarrow{\epsilon}_{\mathcal{E}} s_2$ with $s_2 = s_1$. Since r renames both facts $\text{ind}(rid, x)$ and $\text{out}_{ind}(x)$ to $K(x)$ and $(s_1, s'_1) \in \mathcal{R}$, we have $K(x) \in s_{1,e}$. Additionally, the multiset minus and multiset union operations cancel out after applying r such that $s_{2,e} = s_{1,e}$. Therefore, $(s_2, s'_2) \in \mathcal{R}$.
- $F = \lambda_{\text{in}_{ind}}$: This case is analogous to $F = \lambda_{\text{out}_{ind}}$.
- $F = \lambda_{F_{ind}}$: By definition of F^s and F^e , we have

$$\begin{aligned} \text{Fr}(x \in \text{fresh}) &\in s'_{1,e}, \\ s'_{2,e} &= s'_{1,e} \setminus^m \{\text{Fr}(x)\}, \text{ and} \\ s'_{2,ind,i,rid} &= s'_{1,ind,i,rid} \cup^m \{\text{ind}(rid, x)\}. \end{aligned}$$

Since $(s_1, s'_1) \in \mathcal{R}$, we obtain $\text{Fr}(x) \in s_{1,e}$ enabling us to apply the attacker's message deduction rule (from MD_{Σ}) $[\text{Fr}(x \in \text{fresh})] \Downarrow [K(x)]$, which results in $s_{2,e} = s_{1,e} \setminus^m \{\text{Fr}(x)\} \cup^m \{K(x)\}$. Due to the renaming function r applied to $s'_{2,ind,i,rid}$, we obtain $(s_2, s'_2) \in \mathcal{R}$.

- $e = \chi(e, e')$:
We consider the following four subcases based on the definition of χ :

- $e = \chi(\lambda_{F,i,rid}^s(\bar{m}), \lambda_{F,i,rid}^e(\bar{m}))$ for some $F, i, rid, \bar{m}$:

By definition, neither $\mathcal{R}_{ind}(rid)$ nor $\mathcal{R}_{io}^+$ contain any transition rule with a matching transition label. Hence, this transition step synchronizes a step in $\mathcal{R}_{role}^i$ and $\mathcal{R}_{env}^e$. By definition of our composition operators and since $s'_1 \xrightarrow{e} s'_2$, we have

$$\begin{array}{ccc} s'_{1,i,rid} & \xrightarrow{\lambda_{F,i,rid}^s(\bar{m})} \mathcal{R}_{role}^i(rid) & s'_{2,i,rid} \\ s'_{1,e} & \xrightarrow{\lambda_{F,i,rid}^e(\bar{m})} \mathcal{R}_{env}^e & s'_{2,e} \end{array}$$

and

$$\begin{aligned} s'_{2,j,rid'} &= s'_{1,j,rid'} \\ s'_{2,ind,i,rid} &= s'_{1,ind,i,rid} \\ s'_{2,ind,j,rid'} &= s'_{1,ind,j,rid'} \end{aligned}$$

for all $(j, rid') \neq (i, rid)$. Since the renaming function r behaves like the identity function for facts occurring in the premise and conclusion of rules $\lambda_{F,i,rid}^s(\bar{m})$ and $\lambda_{F,i,rid}^e(\bar{m})$, the same rules can be applied in the abstract states $s_{1,i,rid}$ and $s_{1,e}$. I.e., we have

$$\begin{array}{ccc} s_{1,i,rid} & \xrightarrow{\lambda_{F,i,rid}^s(\bar{m})} \mathcal{R}_{role}^i(rid) & s_{2,i,rid} \\ s_{1,e} & \xrightarrow{\lambda_{F,i,rid}^e(\bar{m})} \mathcal{R}_{env}^e & s_{2,e} \end{array}$$

and $(s_2, s'_2) \in \mathcal{R}$.

- $e = \chi(e', skip)$ for some $e' \in \mathcal{R}_{role}^i(rid)$:
Then, $e' = e$ and by definition of our composition operators, we obtain $s'_{1,i,rid} \xrightarrow{e} \mathcal{R}_{role}^i(rid) s'_{2,i,rid}$ and

$$\begin{aligned} s'_{2,j,rid'} &= s'_{1,j,rid'} \\ s'_{2,ind,i,rid} &= s'_{1,ind,i,rid} \\ s'_{2,ind,j,rid'} &= s'_{1,ind,j,rid'} \\ s'_{2,e} &= s'_{1,e} \end{aligned}$$

for all $(j, rid') \neq (i, rid)$. Since $(s_1, s'_1) \in \mathcal{R}$, we further have $s_{1,i,rid} = s'_{1,i,rid}$,

$s_{1,i,rid} \xrightarrow{e} \mathcal{R}_{role}^i(rid) s_{2,i,rid}$, and, thus, $s_{2,i,rid} = s'_{2,i,rid}$. Therefore, $(s_2, s'_2) \in \mathcal{R}$.

- $e = \chi(e', skip)$ for some $e' \in \mathcal{R}_{ind}(rid)$:
 $e' \neq F^s(rid, x)$ for $F \in \{\lambda_{out,ind}, \lambda_{in,ind}, \lambda_{F,ind}\}$ by definition of χ^+ . Therefore, e' must be the transition adding a public constant or applying a k-ary function f to the state of $\mathcal{R}_{ind}(rid)$. We can simulate either transition in the abstract system $\mathcal{E}$ by performing the corresponding message deduction rule in MD_Σ , which updates the abstract state in the same way after merging the states of the environment and of the components performing protocol-independent operations and applying the renaming function r . Thus, $(s_2, s'_2) \in \mathcal{R}$.
- $e = \chi(skip, e')$ for some $e' \in \mathcal{R}_{env}^e$:
Then, $e' = e$ and, by definition of the composition operators, we obtain $s'_{1,e} \xrightarrow{e} \mathcal{R}_{env}^e s'_{2,e}$, $s'_{2,i,rid} = s'_{1,i,rid}$, and $s'_{2,ind,i,rid} = s'_{1,ind,i,rid}$

for all i, rid . By definition of χ^+ , e cannot have the shape $F^e(rid, x)$ for some rid, x , and $F \in \{\lambda_{out,ind}, \lambda_{in,ind}, \lambda_{F,ind}\}$, which rules out the transitions in $\mathcal{R}_{io}^+$. Thus, $e \in \mathcal{R}_{env}^e$. Since $(s_1, s'_1) \in \mathcal{R}$, we have $s'_{1,e} \subseteq^m s_{1,e}$. Since e 's guard is stable under supermultiset, the rewrite rule e can be applied in state $s_{1,e}$, i.e., $s_{1,e} \xrightarrow{e} \mathcal{R}_{env}^e s_{2,e}$. Since this abstract transition only modifies the submultiset $s'_{1,e}$ by adding or removing facts for which the renaming function r behaves as the identity function and leaves all other $s_{1,i,rid}$ and $s_{1,e} \setminus^m s'_{1,e}$ unchanged, we obtain $s_{2,e} = r((\cup_{i,rid}^m s'_{1,ind,i,rid}) \cup^m s'_{2,e})$. Thus, $s_1 \xrightarrow{e} s_2$ and $(s_2, s'_2) \in \mathcal{R}$. $\square$

Theorem 1 (Soundness). *Suppose Asm. 1 holds and that we have verified, for each role i , the Hoare triple $\vdash_{\pi_{ext}} [\psi_i(rid)] \ c_i(rid) \ [\text{true}]$. Then*

$$(\|_{i,rid} \ \pi_{int}(\mathcal{C}_i(rid))) \parallel_{\chi'} \mathcal{E} \preceq_t \mathcal{R}.$$

Thm. 1 states that composing unboundedly-many instances of each role's LTS $\mathcal{C}_i(rid)$ with the concrete environment $\mathcal{E}$ refines the protocol model $\mathcal{R}$. While this theorem is identical to [52, Theorem 2], the proof differs since our Asm. 1 considers a larger set of traces per LTS $\mathcal{C}_i(rid)$.

Proof. We decompose the proof into a similar series of trace inclusions as [52] but add an additional trace inclusion to abstract the protocol-independent I/O operations to the environment, which contains the DY attacker (cf. Lemma 1).

The first trace inclusion is

$$\begin{aligned} & \left(\|_{i,rid} \ \pi_{int}(\mathcal{C}_i(rid)) \right) \parallel_{\chi'} \mathcal{E} \\ & \preceq \left(\|_{i,rid} \ \pi(\pi'_{ext}(\mathcal{C}_i(rid))) \right) \parallel_{\chi^+} \pi'_{ext}(\pi'_{ext}(\mathcal{E})), \end{aligned} \quad (1)$$

where we obtain the first line from the second by pushing the relabeling $\pi_{ext} \circ \pi'_{ext}$ into the parallel composition, thus changing the set of synchronization labels from χ^+ to χ' .

By combining Asm. 1, the assumption $\vdash_{\pi'_{ext}} [\psi_i(rid)] \ c_i(rid) \ [\text{true}]$, and [52, Theorem 1], we obtain

$$\pi(\pi'_{ext}(\mathcal{C}_i(rid))) \preceq \mathcal{R}_{role}^i(rid) \parallel \mathcal{R}_{ind}(rid), \quad (2)$$

where $\mathcal{R}_{ind}(rid)$ is a multiset rewrite system (MSR) capturing the execution of protocol-independent I/O operations. All facts in this MSR are instantiated with the thread id rid , which helps in distinguishing the facts belonging to each $\mathcal{R}_{ind}$ instance. Additionally, (2) implicitly specifies that the MSRs $\mathcal{R}_{role}^i(rid)$ and $\mathcal{R}_{ind}(rid)$ operate independently, i.e., on different multisets of facts. By performing the taint analysis, we ensure that $\mathcal{R}_{role}^i(rid)$ does not influence $\mathcal{R}_{ind}(rid)$. Checking the opposite, i.e., that $\mathcal{R}_{ind}(rid)$ does not influence $\mathcal{R}_{role}^i(rid)$ by performing a second taint analysis is not necessary. We explicitly track throughout code-level verification the multiset of facts representing the state of $\mathcal{R}_{role}^i(rid)$, which is only manipulated by internal and I/O library functions corresponding to state updates permitted by $\mathcal{R}_{role}^i(rid)$. Therefore, this state cannot be influenced by $\mathcal{R}_{ind}(rid)$.

We can leverage a general composition theorem [53, Theorem 2.3] that implies that trace inclusion is compositional for a large class of composition operators including $|||$ and $||_{\Delta}$. Applying this theorem to (2) and [52, Proposition 2] establishes the trace inclusion

$$\begin{aligned} & \left(|||_{i, rid} \pi(\pi'_{\text{ext}}(\mathcal{C}_i(rid))) \right) ||_{\chi^+} \pi'_{\text{ext}}(\mathcal{C}) \\ & \preceq \left(|||_{i, rid} (\mathcal{R}_{\text{role}}^i(rid) ||| \mathcal{R}_{\text{ind}}(rid)) \right) ||_{\chi^+} \mathcal{R}_{\text{env}}^{e+}. \end{aligned} \quad (3)$$

Applying Lemma 1 in connection with [52, Lemma 1 & Lemma 2] completes the proof. $\square$

B.2. Combining Auto-Active Verification and Static Analyses

In this subsection, we sketch soundness of our combination of auto-active program verification and fully automatic static analyses by constructing a proof in concurrent separation logic [23], [54] for the entire codebase. More specifically, we give an invariant that is maintained by each statement in our programming language (Sec. B.2.1) and present proof rules that use besides certain side conditions only this invariant in their premises and conclusions (Sec. B.2.2). Therefore, we can compose these proof rules to prove $\vdash [\phi] c [\text{true}]$ for an I/O specification ϕ and an entire codebase c . We use DIODON's static analyses to discharge the resulting side conditions (Sec. B.2.3).

To focus on the main proof insights, we deliberately keep the considered programming language simple (cf. Def. 6) and consider the case where an execution of the codebase c corresponds to at most one execution of a protocol role, which is represented by the I/O specification ϕ . We discuss extensions lifting these restrictions in Sec. B.2.4.

Prerequisites. We consider an imperative, concurrent, and heap-manipulating programming language as shown in Def. 6. For simplicity, we omit function boundaries and statements creating complex control flow. Furthermore, we assume that programs are in static single assignment (SSA) form such that we do not have to consider variable reassignments for the purpose of our proof. Besides statements to allocate, read, and write a heap location, we make each auto-actively verified API function of the CORE a dedicated statement in the language even though these statements are themselves implemented as sequences of statements, which are considered by our static analyses. $c := \text{CoreAlloc}(\bar{e})$ corresponds to calling the CORE's constructor and creating a new CORE instance c . $\bar{r} := \text{CoreApi_k}(c, \bar{e})$ represents invoking the k -th API function³ on a CORE instance c using input arguments $\bar{e}$ and return arguments $\bar{r}$. $s_1; s_2$ denotes standard sequential composition of two statements and $\text{fork } (\bar{x}) \{s\}$ spawns a new thread executing statement s while passing variables $\bar{x}$ to this thread. We syntactically require that the newly spawned thread accesses only its own local variables and variables $\bar{x}$.

3. We assume the existence of some total order for API functions, e.g., based on their declarations' syntactical ordering.

Definition 6 (Basic Programming Language). *We consider the following programming language, where S ranges over labeled statements, x over variables, ℓ over statement labels, and e over expressions. We have the usual side effect-free expressions. We use $\bar{y}$ as a shorthand notation to denote lists of kind y .*

$$\begin{aligned} S & \triangleq U^\ell \\ U & \triangleq \text{skip} \mid x := \text{new}() \mid x := *e \mid *x := e \mid \\ & c := \text{CoreAlloc}(\bar{e}) \mid \bar{r} := \text{CoreApi_k}(c, \bar{e}) \mid \\ & S; S \mid \text{fork } (\bar{x}) \{S\} \end{aligned}$$

We call S ; S and $\text{fork } (\bar{x}) \{S\}$ compound statements, while all other statements in our language are called simple. When not relevant, we omit a statement's label ℓ , which uniquely identifies the statement in the program text. We use these labels to refer to the program points before and after each labeled statement. We abstractly treat $c := \text{CoreAlloc}(\bar{e})$ and $\bar{r} := \text{CoreApi_k}(c, \bar{e})$ as first-class statements in our language despite being implemented as sequences of statement that are considered by our static analyses and the auto-active program verifier. This is possible because we can treat these statement as opaque boxes from a proof construction point of view as we prove a Hoare triple for each such statement using the auto-active program verifier.

We assume that all memory accesses in the unverified APPLICATION do not cause data races such that we can reason about their effects. While we could have avoided this assumption by defining that all heap operations in our language are atomic, we try to stay faithful to most programming languages, which specify data races to cause undefined behavior, thus, making this assumption necessary.

Assumption 2 (Data Race Freedom). *We assume that all heap accesses within the APPLICATION, $x := *e$ and $*x := e$, are data race free. I.e., all accesses to the heap locations to which e and x , respectively, point are linearizable and, thus, do not cause data races. Like in Go, we assume that these heap locations are allocated meaning that dereferencing nil is defined behavior and results in a crash.*

Note that this assumption only applies to heap accesses within the APPLICATION as we auto-actively prove that the CORE is memory safe.

By auto-actively verifying the CORE, we prove a Hoare triple for each API function. This allows us to abstractly treat each API function as a statement in our language as long as there are no callbacks; we discuss callbacks in Sec. B.2.4. We syntactically restrict the specification of CORE API functions, i.e., the assertions occurring in the auto-actively verified Hoare triples, such that we can discharge the side conditions using static analyses and, thus, construct a proof for the entire codebase. We state these restrictions immediately after introducing some notational conventions.

Definition 7 (Notation). *We introduce the following notation to simplify forthcoming definitions, explanations, and proofs. $\text{acc}_{\text{nil}}(x)$ denotes full permission for the heap location*

to which x points but only if x is non-**nil**. Analogously, we define $\mathbf{inv}_{\mathbf{nil}}(x)$ for the **CORE** invariant. Lastly, we lift permissions for a heap location to lists thereof, internally using the iterated separating conjunction $\star_i$ ranging over i .

$$\begin{aligned}\mathbf{acc}_{\mathbf{nil}}(x) &\triangleq x \neq \mathbf{nil} \implies \mathbf{acc}(x) \\ \mathbf{inv}_{\mathbf{nil}}(x) &\triangleq x \neq \mathbf{nil} \implies \mathbf{inv}(x) \\ \mathbf{acc}(\bar{x}) &\triangleq \star_{0 \leq i < \text{len}(\bar{x})} \mathbf{acc}(\bar{x}[i]) \\ \mathbf{acc}_{\mathbf{nil}}(\bar{x}) &\triangleq \star_{0 \leq i < \text{len}(\bar{x})} \mathbf{acc}_{\mathbf{nil}}(\bar{x}[i])\end{aligned}$$

where $\text{len}(\bar{x})$ returns the length of list $\bar{x}$ and $\bar{x}[i]$ the i -th element therein.

Assumption 3 (Syntactic Restrictions for **CORE** Specification). We make the following syntactical assumptions about the pre- and postconditions of **CORE** API functions, which ultimately enable us to apply static analyses.

$$\begin{aligned}P_{\text{CoreAlloc}}(\bar{e}) &\triangleq \phi \star R \\ Q_{\text{CoreAlloc}}(c, \bar{e}) &\triangleq \mathbf{inv}(c) \star R' \\ P_{\text{CoreApi}_k}(c, \bar{e}) &\triangleq \mathbf{inv}_{\mathbf{nil}}(c) \star S_k \\ Q_{\text{CoreApi}_k}(c, \bar{e}, \bar{r}) &\triangleq \mathbf{inv}_{\mathbf{nil}}(c) \star S'_k\end{aligned}$$

where R, R', S_k , and S'_k are separation logic assertions that specify permissions for the arguments $\bar{e}$ and, if applicable, $\bar{r}$. Preconditions are free of functional properties and specify at most permissions for non-**nil** arguments, i.e., $\mathbf{acc}_{\mathbf{nil}}(\bar{e}) \models R$ and $\mathbf{acc}_{\mathbf{nil}}(\bar{e}) \models S_k$. Each postcondition needs to specify the same or more permissions than the respective precondition, i.e., $R' \models R$ and $S'_k \models S_k$. Additionally, postconditions need to specify full permission to every heap location that becomes accessible to the **APPLICATION** and that is created within the corresponding **CORE** function or any function transitively called thereby. For simplicity, we disallow **CoreAlloc**($\bar{e}$) to return such heap locations other than the **CORE** instance itself and, thus, permissions to such heap locations can only occur in S'_k for the return arguments $\bar{r}$. Furthermore, we restrict the input arguments $\bar{e}$ and output arguments $\bar{r}$ to be shallow, i.e., their transitive closure of reachable heap locations is the singleton set, i.e., $\forall e \in \bar{e}. e \neq \mathbf{nil} \implies \text{reach}(e) = \{e\}$ and analogously for $\bar{r}$. This restriction simplifies the reasoning about which heap locations are passed between the **CORE** and **APPLICATION**. However, lifting this restriction is possible and would require that S_k and S'_k specify the permission for every reachable heap location.

B.2.1. Program invariant. In order to define composable proof rules for our language, we define a program invariant that is maintained by each statement. Our invariant conceptually partitions the heap among two dimensions, namely whether a heap location is accessible by multiple threads and whether a heap location is owned by the **APPLICATION** as opposed to the **CORE**. As we will formalize later, we call a heap location h **APPLICATION**-managed if h is under the **APPLICATION**'s control, which means that it is not covered by the **CORE** invariant. Furthermore, we ensure that the **APPLICATION** only accesses memory that is **APPLICATION**-managed.

We make the heap partitioning explicit by introducing ghost variables tracking the heap locations belonging to each partition. We use a global ghost set pointed to by sgh tracking the set of heap locations that are accessible by multiple threads. The thread-local ghost variable slh tracks **APPLICATION**-managed heap locations that are only accessible by the current thread. Lastly, the thread-local variable sih tracks the **CORE** instance if it is already allocated.

Relying on these ghost variables, we can define the program invariants Π_l and Π_g that specify the separation logic permissions held by a thread at each program point, as shown in Def. 8, where used is a pointer to a boolean specifying whether the I/O permissions ϕ have already been consumed to allocate a **CORE** instance.

Definition 8 (Program Invariants).

$$\begin{aligned}\Pi_l &\triangleq (\star_{l \in \text{slh}} \mathbf{acc}(l)) \star (\star_{i \in \text{sih}} \mathbf{inv}(i)) \\ \Pi_g &\triangleq \mathbf{acc}(\text{sgh}) \star (\star_{g \in * \text{sgh}} \mathbf{acc}(g)) \star \\ &\quad \mathbf{acc}(\text{used}) \star (\neg(* \text{used}) \implies \phi)\end{aligned}$$

Π_l specifies permissions that are exclusively owned by each thread. The first conjunct specifies (full) permission to every heap location in slh , which, as we will see, holds every heap location that is only accessible by the current thread and is unrelated to **CORE** instances. Additionally, Π_l specifies that the **CORE** invariant $\mathbf{inv}(i)$ holds for each **CORE** instance i . Note that $\mathbf{inv}(i)$ is a separation logic predicate that specifies permissions for a subset of the transitively reachable heap locations starting from i and possibly functional properties about these heap locations. While the definition of $\mathbf{inv}(i)$ matters for the **CORE**'s auto-active verification, we treat $\mathbf{inv}(i)$ for the purpose of the program invariant as an opaque separation logic resource.

Π_g specifies permissions to heap locations that are potentially shared among multiple threads. When accessing such a heap location, a thread can temporarily acquire the corresponding permission from Π_g , which is justified as long as all accesses to this location are linearizable. Since we assume absence of data races (cf. Asm. 2), there exists a linearization of heap accesses such that permission for manipulating g , i.e., $\mathbf{acc}(g)$, can temporarily be obtained from Π_g for the manipulation's duration. Furthermore, Π_g specifies the I/O permissions ϕ if they have not been used yet to create a **CORE** instance, in which case the pointer used points to a heap location storing the value false. As mentioned, we focus in this proof on the case of creating at most one **CORE** instance. However, this conjunct can easily be adapted to provide a family of I/O permissions such that the creation of arbitrarily-many **CORE** instances becomes possible, as we will detail in Sec. B.2.4.

Since the presented program invariants rely on ghost variables to specify permissions, we have to ensure that these ghost variables stay in sync with a program's execution, i.e., the effects of each statement. Hence, we present algorithm $\mathbb{A}$ in Fig. 11 that augments a program with ghost statements updating the ghost variables according to each statement's effects. These ghost statements manipulate only

$$\begin{aligned}
& \mathbb{A}(\text{skip}) \rightsquigarrow \text{skip} \\
& \mathbb{A}(x := \text{new}()) \rightsquigarrow x := \text{new}(); \text{slh} := \text{slh} \cup_{\text{nil}} \{x\} \\
& \mathbb{A}(x := *e) \rightsquigarrow \begin{cases} \text{slh} := \text{slh} \setminus \{e\}; x := *e; \text{slh} := \text{slh} \cup_{\text{nil}} \{e\} & \text{if } e \in \text{slh} \\ \text{atomic} \{ * \text{sgh} := * \text{sgh} \setminus \{e\}; x := *e; * \text{sgh} := * \text{sgh} \cup_{\text{nil}} \{e\} \} & \text{otherwise} \end{cases} \\
& \mathbb{A}(*x := e) \rightsquigarrow \begin{cases} \text{slh} := \text{slh} \setminus \{x\}; *x := e; \text{slh} := \text{slh} \cup_{\text{nil}} \{x\} & \text{if } x \in \text{slh} \\ \text{atomic} \{ * \text{sgh} := * \text{sgh} \setminus \{x\}; \text{slh} := \text{slh} \setminus (\text{reach}(e) \cap \text{slh}); & \text{otherwise} \\ *x := e; * \text{sgh} := * \text{sgh} \cup_{\text{nil}} \{x\} \cup (\text{reach}(e) \cap \text{slh}) \} & \end{cases} \\
& \mathbb{A}(c := \text{CoreAlloc}(\bar{e})) \rightsquigarrow \text{atomic} \{ * \text{used} := \text{true} \}; \text{slh} := \text{slh} \setminus \bar{e}; c := \text{CoreAlloc}(\bar{e}); \text{slh} := \text{slh} \cup_{\text{nil}} \bar{e}; \text{sih} := \text{sih} \cup_{\text{nil}} \{c\} \\
& \mathbb{A}(\bar{r} := \text{CoreApi_k}(c, \bar{e})) \rightsquigarrow \text{sih} := \text{sih} \setminus \{c\}; \text{slh} := \text{slh} \setminus \bar{e}; \bar{r} := \text{CoreApi_k}(c, \bar{e}); \text{slh} := \text{slh} \cup_{\text{nil}} \bar{e} \cup \bar{r}; \text{sih} := \text{sih} \cup_{\text{nil}} \{c\} \\
& \mathbb{A}(s_1; s_2) \rightsquigarrow \mathbb{A}(s_1); \mathbb{A}(s_2) \\
& \mathbb{A}(\text{fork}(\bar{x}) \{s\}) \rightsquigarrow \text{slh} := \text{slh} \setminus (\text{reach}(\bar{x}) \cap \text{slh}); * \text{sgh} := * \text{sgh} \cup_{\text{nil}} (\text{reach}(\bar{x}) \cap \text{slh}); \text{fork}(\bar{x}) \{ \text{slh} := \emptyset; \text{sih} := \emptyset; \mathbb{A}(s) \}
\end{aligned}$$

Figure 11. Algorithm $\mathbb{A}$ transforms a codebase by inserting ghost statements. We define this algorithm by cases, i.e., describe how $\mathbb{A}$ transforms each statement s to a statement s' , written as $\mathbb{A}(s) \rightsquigarrow s'$. $\text{reach}(e)$ computes the set of transitively reachable heap locations from expression e . The set union operation ignores nil , as variables might be nil , i.e., $S_1 \cup_{\text{nil}} S_2 \triangleq (S_1 \cup S_2) \setminus \text{nil}$. This ensures that nil is never contained in any ghost set.

ghost variables and aid verification without changing the input program's runtime behavior. Thus, these ghost variables and ghost statements can be erased before compilation.

Common to all cases of algorithm $\mathbb{A}$ is that for a statement s , first, the current heap is partitioned into a heap H on which s possibly operates and the remaining heap F that s leaves untouched by removing the separation logic resources for H via corresponding ghost set subtractions. The separation logic resources belonging to heap F remain in the ghost sets. Afterwards, statement s is executed that changes heap H to H' , followed by merging the heaps H' and F via ghost set union operations.

Allocating a heap location operates on an empty heap and produces a new heap location, which is added to the set of local heap locations as there is no way any other thread might already have gained access thereto. Dereferencing a pointer e and reading the corresponding heap location requires a permission for the duration of this operation. Therefore, we first subtract and afterwards add this location from the current heap by manipulating either the ghost set of local or global heap locations depending on whether this heap location is contained in slh . In case the heap location is in the ghost set of global heap locations, we insert an atomic block, which is justified by Asm. 2 stating that accesses to this heap location are linearizable.

Noteworthy are write operations to heap locations, especially in the case that a heap location is accessible by other threads as the written value becomes accessible by these threads. Hence, we first remove all local heap locations that are transitively reachable from the written value and add them afterwards to the ghost set of global heap location as these locations possibly escape the current thread via this write operation. Similarly when forking a thread, the heap locations that are reachable from the variables $\bar{x}$ escape the current thread and, thus, the sets of local and global heap locations are updated accordingly.

For $\text{CoreAlloc}(\bar{e})$ and $\bar{r} := \text{CoreApi_k}(c, \bar{e})$, the algorithm $\mathbb{A}$ adds and subtracts only $\bar{e}$ and $\bar{r}$ as opposed to all transitively reachable heap locations. This is sufficient because Asm. 3 restricts $\bar{e}$ and $\bar{r}$ to be shallow and, thus, no other heap locations are reachable. However, extending algorithm $\mathbb{A}$ to support non-shallow arguments would be

straightforward by adding and removing $\bar{e}$'s and $\bar{r}$'s transitive closure to and from slh , respectively.

B.2.2. Proof Rules. Thanks to the program invariants and the ghost statements that algorithm $\mathbb{A}$ inserts into a program, we can define proof rules as shown in Fig. 12. In particular, all proof rules share the same pre- and postcondition, namely the local and global program invariants Π_l and Π_g , resp., which allow us to compose the proof rules to obtain a whole program proof. The proof rules' simplicity is enabled by their side conditions (cf. Fig. 13) that we discharge using our static analyses.

Besides containment of heap locations in particular ghost sets, the side conditions rely on disjointness of input arguments, which we formally define next. Informally, two arguments are disjoint if they point to different heap locations or one of the arguments is nil .

Definition 9 (Variable value). $\text{val}_\tau(x)$ denotes the value of variable x on trace τ . Since we assume that our programs are in SSA-form, this definition is independent of a particular program point. However, x must be declared such that $\text{val}_\tau(x)$ is defined.

Definition 10 (Disjointness). Two pointer variables x and y are disjoint if their pointer value is different or nil for all traces τ .

$$\text{disjoint}(\{x, y\}) \triangleq \forall \tau. \text{val}_\tau(x) = \text{nil} \vee \text{val}_\tau(x) \neq \text{val}_\tau(y)$$

We straightforwardly lift this definition to lists of variables $\bar{z}$, where $\text{disjoint}(\bar{z})$ denotes pairwise disjointness between every element in $\bar{z}$.

Next, we sketch the proof rules' soundness proof, which relies on the side conditions ω . Afterwards, we define what properties our static analyses provide given that their execution succeeded and show that these properties imply the side conditions ω . We conclude by proving a corollary stating that we construct a Hoare triple for the entire codebase.

Theorem 2 (Soundness of proof rules).

$$\text{If } \Pi_g \vdash [\Pi_l] \mathbb{A}(s) [\Pi_l], \text{ then } \Pi_g \models [\Pi_l] \mathbb{A}(s) [\Pi_l]$$

$$\begin{array}{c}
\frac{\omega(s_{\text{simple}})}{\Pi_g \vdash [\Pi_l] \mathbb{A}(s_{\text{simple}}) [\Pi_l]} \text{ (SIMPLE)} \quad \frac{\Pi_g \vdash [\Pi_l] \mathbb{A}(s_1) [\Pi_l] \quad \Pi_g \vdash [\Pi_l] \mathbb{A}(s_2) [\Pi_l]}{\Pi_g \vdash [\Pi_l] \mathbb{A}(s_1; s_2) [\Pi_l]} \text{ (SEQ)} \quad \frac{\Pi_g \vdash [\Pi_l] \mathbb{A}(s) [\Pi_l]}{\Pi_g \vdash [\Pi_l] \mathbb{A}(\text{fork } (\bar{x}) \{s\}) [\Pi_l]} \text{ (FORK)}
\end{array}$$

Figure 12. Proof rules where s_{simple} ranges over all *simple* statements and s , s_1 , and s_2 range over all statements. ω denotes a statement's side conditions (cf. Fig. 13).

$$\begin{aligned}
\omega(x := *e) &\triangleq e \in \text{slh} \cup *sgh \\
\omega(*x := e) &\triangleq x \in \text{slh} \cup *sgh \\
\omega(c := \text{CoreAlloc}(\bar{e})) &\triangleq * \text{used} = \text{false} \wedge \\
&\quad (\text{set}(\bar{e}) \setminus \text{nil}) \subseteq \text{slh} \wedge \\
&\quad \text{disjoint}(\bar{e}) \\
\omega(\bar{r} := \text{CoreApi_k}(c, \bar{e})) &\triangleq (\text{set}(\bar{e}) \setminus \text{nil}) \subseteq \text{slh} \wedge \\
&\quad \text{disjoint}(\bar{e}) \wedge \\
&\quad (c \in \text{slh} \vee c = \text{nil})
\end{aligned}$$

Figure 13. Side conditions for our statements, which are amenable to static analyses. ω evaluates to true for all statements not listed above and $\text{set}(l)$ returns the set of elements in list l . We implicitly refer to variables' values, e.g., $v \in S$ denotes that the value of variable v is contained in set stored in variable S as opposed to the variables' syntactical representation.

Proof sketch. We perform structural induction over the input statement s to algorithm $\mathbb{A}$ and construct a proof tree in separation logic building up on the proof rules by Vafeiadis [54]. We use small caps font to denote proof rules, such as **SKIP**. All rules in this theorem's proof are from Vafeiadis [54], except **FORK** and **SEQ*** that are straightforward extensions from the parallel and sequential composition rules, respectively. Side conditions arising in the proof trees are marked in blue and form ω (cf. Fig. 13).

- $\mathbb{A}(\text{skip})$: Since the algorithm $\mathbb{A}$ does not insert any ghost commands and **skip** does not alter the program state, Π_l is trivially maintained. The **SKIP** rule is immediately applicable and completes the proof tree.
- $\mathbb{A}(x := \text{new}())$: Fig. 23 shows the proof tree that uses Fig. 14 as a sub-proof for inserting a heap location into the ghost set of local heap locations.
- $\mathbb{A}(x := *e)$: The side condition ω ensures that $e \in \text{slh} \cup *sgh$ holds. If $e \in \text{slh}$ then Fig. 24 is a valid proof tree for this read operation. Otherwise, $e \in *sgh$ holds and Fig. 25 shows the corresponding proof tree.
- $\mathbb{A}(*x := e)$: For write operations, we construct a proof tree similar to read operations, as explained in the case above, except that we extract permissions for x instead of e from the program invariants and replace applications of the **READ** rule by **WRITE**. We can apply these rules because we possess full permission (as opposed to only partial permission) to the heap location (i.e., $\text{acc}(x)$).
- $\mathbb{A}(c := \text{CoreAlloc}(\bar{e}))$: Fig. 27 shows the proof tree extending the subproof that the auto-active program verifier implicitly constructs (in Fig. 26) while verifying the Hoare triple for **CoreAlloc**($\bar{e}$).
- $\mathbb{A}(\bar{r} := \text{CoreApi_k}(c, \bar{e}))$: We construct a proof tree in Fig. 29 using Fig. 28 as a subtree that is similar

to the one for the **CORE** allocation command with the main difference that the precondition requires $\text{inv}_{\text{nil}}(c)$ instead of the I/O permissions ϕ . The side condition $c \in \text{slh} \vee c = \text{nil}$ ensures that we can obtain $\text{inv}_{\text{nil}}(c)$ from Π_l within the proof.

- $\mathbb{A}(s_1; s_2)$: We apply the standard **SEQ** rule from separation logic to combine the proof subtrees for $\mathbb{A}(s_1)$ and $\mathbb{A}(s_2)$ that we obtain by applying our induction hypothesis.
- $\mathbb{A}(\text{fork } (\bar{x}) \{s\})$: Fig. 31 shows the proof tree that applies the induction hypothesis to $\mathbb{A}(s)$. Since the algorithm $\mathbb{A}$ removes the permissions for heap locations only in $\text{reach}(\bar{x}) \cap \text{slh}$, the resulting side condition $(\text{reach}(\bar{x}) \cap \text{slh}) \subseteq \text{slh}$ is trivial since these heap locations are by definition contained in slh .

The main proof insight is that we ensure that the global invariant covers the permissions for all heap locations that become accessible by the spawned thread and establish the local invariant for the spawned thread by initializing the set of local heap locations and (local) **CORE** instances to the empty set. $\text{reach}(\bar{x})$ forms an upper bound on the heap locations that command s might access because we syntactically require that s accesses only $\bar{x}$ and its own local variables. $\square$

B.2.3. Static Analyses. Since our proof rules rely on the side conditions ω (cf. Fig. 13), we introduce next our static analyses, cover the properties we assume they provide, and show that these properties imply ω . We end by proving a corollary that we can construct a whole program proof for a codebase given that we have auto-actively verified the **CORE** and successfully executed the static analyses.

Pointer Analysis. A pointer analysis computes for each pointer x a set of heap locations L to where x may point, which we formalize as a judgement $\text{pts}(x) = L$. Each heap location in L is identified by its allocation site, which corresponds to the label of a particular statement in the program's text. Note that this analysis over-approximates the set of heap locations that actually change when writing to x . The pointer analysis we are using is context insensitive, i.e., ignores control flow and ordering of statements. Thus, we omit the program location at which such a judgement holds as it holds for all program locations within a given codebase. If necessary, we could employ a context sensitive pointer analysis to increase precision.

To formalize what the pointer analysis computes, let us first state several definitions before stating the pointer analysis' soundness, which we assume.

$$\begin{array}{c}
\frac{}{\Pi_g \vdash [\forall l \in \text{slh} \cup_{\text{nil}} \{x\}. \mathbf{acc}(l)] \text{slh} := \text{slh} \cup_{\text{nil}} \{x\} \quad [\forall l \in \text{slh}. \mathbf{acc}(l)]} \text{ASSIGN} \\
\frac{}{\Pi_g \vdash [(\forall l \in \text{slh}. \mathbf{acc}(l)) \star \mathbf{acc}_{\text{nil}}(x)] \text{slh} := \text{slh} \cup_{\text{nil}} \{x\} \quad [\forall l \in \text{slh}. \mathbf{acc}(l)]} \text{CONSEQ} \\
\frac{}{\Pi_g \vdash [\Pi_l \star \mathbf{acc}_{\text{nil}}(x)] \text{slh} := \text{slh} \cup_{\text{nil}} \{x\} \quad [\Pi_l]} \text{FRAME}
\end{array}$$

Figure 14. Proof tree for $\text{slh} := \text{slh} \cup_{\text{nil}} \{x\}$, where $\mathbf{acc}_{\text{nil}}(e) \triangleq e \neq \text{nil} \implies \mathbf{acc}(e)$.

$$\begin{array}{c}
\frac{}{\mathbf{emp} \vdash [\mathbf{acc}(\text{sgh}) \star \text{sgh} = v] \quad \text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\} \quad [\mathbf{acc}(\text{sgh}) \star \text{sgh} = v \cup_{\text{nil}} \{x\}]} \text{WRITE} \\
\frac{}{\mathbf{emp} \vdash [\mathbf{acc}(\text{sgh}) \star \text{sgh} = v \star R] \quad \text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\} \quad [\mathbf{acc}(\text{sgh}) \star \text{sgh} = v \cup_{\text{nil}} \{x\} \star R]} \text{FRAME} \\
\frac{}{\mathbf{emp} \vdash [\mathbf{acc}(\text{sgh}) \star (\forall g \in \text{sgh}. \mathbf{acc}(g)) \star \mathbf{acc}_{\text{nil}}(x)] \quad \text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\} \quad [\mathbf{acc}(\text{sgh}) \star \forall g \in \text{sgh}. \mathbf{acc}(g)]} \text{CONSEQ} \\
\frac{}{\mathbf{emp} \vdash [\Pi_g \star \mathbf{acc}_{\text{nil}}(x)] \quad \text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\} \quad [\Pi_g]} \text{FRAME} \\
\text{with } R \triangleq \forall g \in (v \cup_{\text{nil}} \{x\}). \mathbf{acc}(g)
\end{array}$$

Figure 15. Proof tree for $\text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\}$ given that Π_g is already local, where v is a *fresh* variable and the WRITE rule has been naturally extended to internally perform a heap read operation returning the value v for sgh as specified in the precondition.

$$\begin{array}{c}
\vdots \text{ Fig. 15} \\
\mathbf{emp} \vdash [\Pi_g \star \mathbf{acc}_{\text{nil}}(x)] \quad \text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\} \quad [\Pi_g] \\
\frac{}{\Pi_g \vdash [\mathbf{acc}_{\text{nil}}(x)] \quad \text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\} \quad [\mathbf{emp}]} \text{ATOM}
\end{array}$$

Figure 16. Proof tree for $\text{sgh} := \text{sgh} \cup_{\text{nil}} \{x\}$.

$$\begin{array}{c}
\frac{}{\Pi_g \vdash [\forall i \in \text{sih} \cup_{\text{nil}} \{c\}. \mathbf{inv}(i)] \text{sih} := \text{sih} \cup_{\text{nil}} \{c\} \quad [\forall i \in \text{sih}. \mathbf{inv}(i)]} \text{ASSIGN} \\
\frac{}{\Pi_g \vdash [(\forall i \in \text{sih}. \mathbf{inv}(i)) \star \mathbf{inv}_{\text{nil}}(c)] \text{sih} := \text{sih} \cup_{\text{nil}} \{c\} \quad [\forall i \in \text{sih}. \mathbf{inv}(i)]} \text{CONSEQ} \\
\frac{}{\Pi_g \vdash [\Pi_l \star \mathbf{inv}_{\text{nil}}(c)] \text{sih} := \text{sih} \cup_{\text{nil}} \{c\} \quad [\Pi_l]} \text{FRAME}
\end{array}$$

Figure 17. Proof tree for $\text{sih} := \text{sih} \cup_{\text{nil}} \{c\}$, where $\mathbf{inv}_{\text{nil}}(c) \triangleq c \neq \text{nil} \implies \mathbf{inv}(c)$.

$$\begin{array}{c}
\frac{}{\Pi_g \vdash [\forall l \in \text{slh} \setminus \{e\}. \mathbf{acc}(l)] \text{slh} := \text{slh} \setminus \{e\} \quad [\forall l \in \text{slh}. \mathbf{acc}(l)]} \text{ASSIGN} \\
\frac{}{\Pi_g \vdash [(\forall l \in \text{slh} \setminus \{e\}. \mathbf{acc}(l)) \star \mathbf{acc}(e)] \text{slh} := \text{slh} \setminus \{e\} \quad [(\forall l \in \text{slh}. \mathbf{acc}(l)) \star \mathbf{acc}(e)]} \text{FRAME} \\
\frac{}{\Pi_g \vdash [\forall l \in \text{slh}. \mathbf{acc}(l)] \text{slh} := \text{slh} \setminus \{e\} \quad [(\forall l \in \text{slh}. \mathbf{acc}(l)) \star \mathbf{acc}(e)]} \text{CONSEQ} \\
\frac{}{\Pi_g \vdash [\Pi_l] \text{slh} := \text{slh} \setminus \{e\} \quad [\Pi_l \star \mathbf{acc}(e)]} \text{FRAME}
\end{array}$$

Figure 18. Proof tree for $\text{slh} := \text{slh} \setminus \{e\}$ if $e \in \text{slh}$.

$$\begin{array}{c}
\frac{}{\Pi_g \vdash [\forall l \in \text{slh} \setminus \{e\}. \mathbf{acc}(l)] \text{slh} := \text{slh} \setminus \{e\} \quad [\forall l \in \text{slh}. \mathbf{acc}(l)]} \text{ASSIGN} \\
\frac{}{\Pi_g \vdash [(\forall l \in \text{slh} \setminus \{e\}. \mathbf{acc}(l)) \star \mathbf{acc}_{\text{nil}}(e)] \text{slh} := \text{slh} \setminus \{e\} \quad [(\forall l \in \text{slh}. \mathbf{acc}(l)) \star \mathbf{acc}_{\text{nil}}(e)]} \text{FRAME} \\
\frac{}{\Pi_g \vdash [\forall l \in \text{slh}. \mathbf{acc}(l)] \text{slh} := \text{slh} \setminus \{e\} \quad [(\forall l \in \text{slh}. \mathbf{acc}(l)) \star \mathbf{acc}_{\text{nil}}(e)]} \text{CONSEQ} \\
\frac{}{\Pi_g \vdash [\Pi_l] \text{slh} := \text{slh} \setminus \{e\} \quad [\Pi_l \star \mathbf{acc}_{\text{nil}}(e)]} \text{FRAME}
\end{array}$$

Figure 19. Alternative proof tree for $\text{slh} := \text{slh} \setminus \{e\}$ that permits e being nil .

$$\begin{array}{c}
\frac{e \in *sgh \quad \vdots \text{Fig. 20} \quad \frac{\text{emp} \vdash [\text{acc}(e)] \quad s_2 \quad [\text{acc}(e)] \quad \text{READ} \quad \text{emp} \vdash [\Pi_g \star \text{acc}_{\text{nil}}(e)] \quad s_3 \quad [\Pi_g] \quad \vdots \text{Fig. 15}}{\text{emp} \vdash [\Pi_g \star \text{acc}(e)] \quad s_2 \quad [\Pi_g \star \text{acc}(e)]} \text{FRAME} \quad \frac{\text{emp} \vdash [\Pi_g \star \text{acc}(e)] \quad s_3 \quad [\Pi_g] \quad \vdots \text{Fig. 15}}{\text{emp} \vdash [\Pi_g \star \text{acc}(e)] \quad s_3 \quad [\Pi_g]} \text{SEQ}^*}{\text{emp} \vdash [\Pi_g] \quad s_1 \quad [\Pi_g \star \text{acc}(e)] \quad \text{emp} \vdash [\Pi_g \star \text{acc}(e)] \quad s_2 \quad [\Pi_g \star \text{acc}(e)] \quad \text{emp} \vdash [\Pi_g \star \text{acc}(e)] \quad s_3 \quad [\Pi_g]} \text{SEQ}^* \\
\frac{\text{emp} \vdash [\Pi_g] \quad s_1; s_2; s_3 \quad [\Pi_g]}{\Pi_g \vdash [\text{emp}] \quad \text{atomic} \{s_1; s_2; s_3\} \quad [\text{emp}]} \text{ATOM} \\
\frac{\Pi_g \vdash [\text{emp}] \quad \text{atomic} \{s_1; s_2; s_3\} \quad [\text{emp}]}{\Pi_g \vdash [\Pi_l] \quad \text{atomic} \{s_1; s_2; s_3\} \quad [\Pi_l]} \text{FRAME} \\
\text{with } s_1 \triangleq *sgh := *sgh \setminus \{e\} \quad s_2 \triangleq x := *e \quad s_3 \triangleq *sgh := *sgh \cup_{\text{nil}} \{e\}
\end{array}$$

Figure 25. Proof tree for $\mathbb{A}(x := *e)$ if $e \notin \text{slh}$.

$$\begin{array}{c}
\vdots \text{auto-active verification} \\
\frac{\Pi_g \vdash [P_{\text{CoreAlloc}}(\bar{e})] \quad c := \text{CoreAlloc}(\bar{e}) \quad [Q_{\text{CoreAlloc}}(c, \bar{e})]}{\Pi_g \vdash [P_{\text{CoreAlloc}}(\bar{e}) \star F] \quad c := \text{CoreAlloc}(\bar{e}) \quad [Q_{\text{CoreAlloc}}(c, \bar{e}) \star F]} \text{FRAME} \\
\frac{\text{Asm. 3} \quad \Pi_g \vdash [P_{\text{CoreAlloc}}(\bar{e}) \star F] \quad c := \text{CoreAlloc}(\bar{e}) \quad [Q_{\text{CoreAlloc}}(c, \bar{e}) \star F]}{\Pi_g \vdash [\text{acc}_{\text{nil}}(\bar{e}) \star \phi] \quad c := \text{CoreAlloc}(\bar{e}) \quad [\text{acc}_{\text{nil}}(\bar{e}) \star \text{inv}(c)]} \text{CONSEQ} \\
\frac{\Pi_g \vdash [\text{acc}_{\text{nil}}(\bar{e}) \star \phi] \quad c := \text{CoreAlloc}(\bar{e}) \quad [\text{acc}_{\text{nil}}(\bar{e}) \star \text{inv}(c)]}{\Pi_g \vdash [\Pi_l \star \text{acc}_{\text{nil}}(\bar{e}) \star \phi] \quad c := \text{CoreAlloc}(\bar{e}) \quad [\Pi_l \star \text{acc}_{\text{nil}}(\bar{e}) \star \text{inv}(c)]} \text{FRAME}
\end{array}$$

Figure 26. Proof tree for $c := \text{CoreAlloc}(\bar{e})$ using the subproof that we extract from the auto-active program verifier. The side condition (Asm. 3) states that $P_{\text{CoreAlloc}}(\bar{e}) = \phi \star R$ and $\text{acc}_{\text{nil}}(\bar{e}) \models R$. We call F the permissions that are framed around, i.e., $\text{acc}_{\text{nil}}(\bar{e}) = R \star F$. The side condition further specifies that $Q_{\text{CoreAlloc}}(c, \bar{e}) = \text{inv}(c) \star R'$ and $R' \models R$ hold, allowing us to derive $R' \star F \models \text{acc}_{\text{nil}}(\bar{e})$. Thus, we can apply the CONSEQ rule. We abuse the notation $\text{acc}_{\text{nil}}(\bar{e})$ to denote the iterated separating conjunction expressing $\text{acc}_{\text{nil}}(e)$ for each element e in $\bar{e}$, i.e., $\forall i. 0 \leq i < \text{len}(\bar{e}) \implies \text{acc}_{\text{nil}}(\bar{e}[i])$, where $\text{len}(\bar{e})$ and $\bar{e}[i]$ return the length and the i -th element of the list $\bar{e}$, respectively.

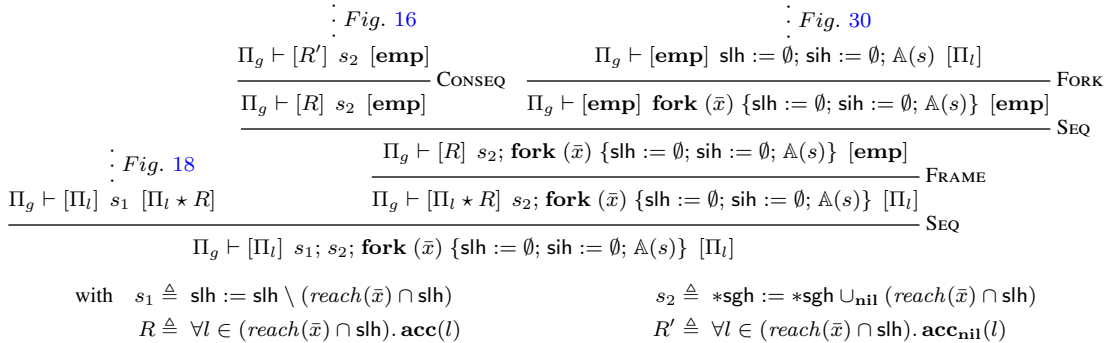
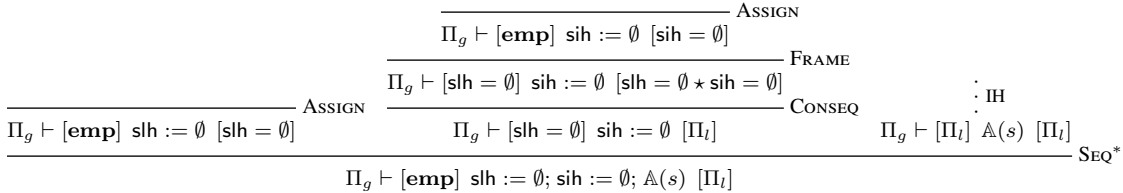
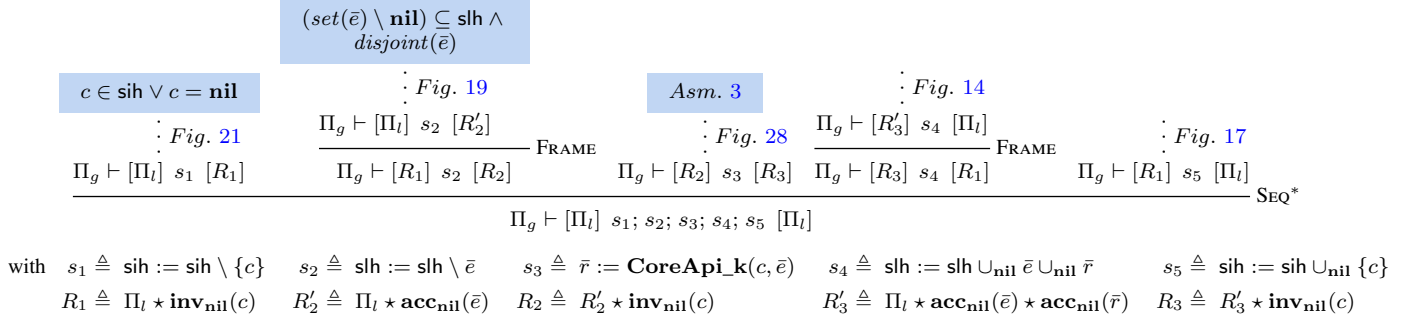
$$\begin{array}{c}
\frac{\neg(*\text{used}) \quad \vdots \text{Fig. 22} \quad \frac{\Pi_g \vdash [\Pi_l] \quad s_1 \quad [\Pi_l \star \phi]}{\Pi_g \vdash [\Pi_l] \quad s_1 \quad [\Pi_l \star \phi]} \quad \frac{\vdots \text{Fig. 19} \quad \frac{\Pi_g \vdash [\Pi_l] \quad s_2 \quad [R'_2]}{\Pi_g \vdash [\Pi_l \star \phi] \quad s_2 \quad [R_2]} \text{FRAME} \quad \frac{\text{Asm. 3} \quad \vdots \text{Fig. 26} \quad \frac{\Pi_g \vdash [R'_2] \quad s_4 \quad [\Pi_l]}{\Pi_g \vdash [R_2] \quad s_3 \quad [R_3]} \text{FRAME} \quad \frac{\vdots \text{Fig. 14} \quad \frac{\Pi_g \vdash [R'_4] \quad s_5 \quad [\Pi_l]}{\Pi_g \vdash [R_4] \quad s_5 \quad [\Pi_l]} \text{FRAME} \quad \frac{\vdots \text{Fig. 17} \quad \frac{\Pi_g \vdash [R'_4] \quad s_5 \quad [\Pi_l]}{\Pi_g \vdash [R_4] \quad s_5 \quad [\Pi_l]} \text{SEQ}}{\Pi_g \vdash [\Pi_l] \quad s_1; s_2; s_3; s_4; s_5 \quad [\Pi_l]} \text{SEQ}^*
\end{array}$$

$$\begin{array}{l}
\text{with } s_1 \triangleq \text{atomic} \{*\text{used} := \text{true}\} \quad s_2 \triangleq \text{slh} := \text{slh} \setminus \bar{e} \quad s_3 \triangleq c := \text{CoreAlloc}(\bar{e}) \quad s_4 \triangleq \text{slh} := \text{slh} \cup_{\text{nil}} \bar{e} \quad s_5 \triangleq \text{slh} := \text{slh} \cup_{\text{nil}} \{c\} \\
R'_2 \triangleq \Pi_l \star \text{acc}_{\text{nil}}(\bar{e}) \quad R_2 \triangleq R'_2 \star \phi \quad R_3 \triangleq R'_2 \star \text{inv}(c) \quad R_4 \triangleq \Pi_l \star \text{inv}(c) \quad R'_4 \triangleq \Pi_l \star \text{inv}_{\text{nil}}(c)
\end{array}$$

Figure 27. Proof tree for $\mathbb{A}(c := \text{CoreAlloc}(\bar{e}))$. We naturally extend Fig. 14 and Fig. 18 to adding and removing *lists* of heap locations to and from the ghost set slh , respectively. The latter requires their disjointness.

$$\begin{array}{c}
\vdots \text{auto-active verification} \\
\frac{\Pi_g \vdash [P_{\text{CoreApi}_k}(c, \bar{e})] \quad \bar{r} := \text{CoreApi}_k(c, \bar{e}) \quad [Q_{\text{CoreApi}_k}(c, \bar{e}, \bar{r})]}{\Pi_g \vdash [P_{\text{CoreApi}_k}(c, \bar{e}) \star F] \quad \bar{r} := \text{CoreApi}_k(c, \bar{e}) \quad [Q_{\text{CoreApi}_k}(c, \bar{e}, \bar{r}) \star F]} \text{FRAME} \\
\frac{\text{Asm. 3} \quad \Pi_g \vdash [P_{\text{CoreApi}_k}(c, \bar{e}) \star F] \quad \bar{r} := \text{CoreApi}_k(c, \bar{e}) \quad [Q_{\text{CoreApi}_k}(c, \bar{e}, \bar{r}) \star F]}{\Pi_g \vdash [\text{inv}_{\text{nil}}(c) \star \text{acc}_{\text{nil}}(\bar{e})] \quad \bar{r} := \text{CoreApi}_k(c, \bar{e}) \quad [\text{inv}_{\text{nil}}(c) \star \text{acc}_{\text{nil}}(\bar{e}) \star \text{acc}_{\text{nil}}(\bar{r})]} \text{CONSEQ} \\
\frac{\Pi_g \vdash [\text{inv}_{\text{nil}}(c) \star \text{acc}_{\text{nil}}(\bar{e})] \quad \bar{r} := \text{CoreApi}_k(c, \bar{e}) \quad [\text{inv}_{\text{nil}}(c) \star \text{acc}_{\text{nil}}(\bar{e}) \star \text{acc}_{\text{nil}}(\bar{r})]}{\Pi_g \vdash [\Pi_l \star \text{inv}_{\text{nil}}(c) \star \text{acc}_{\text{nil}}(\bar{e})] \quad \bar{r} := \text{CoreApi}_k(c, \bar{e}) \quad [\Pi_l \star \text{inv}_{\text{nil}}(c) \star \text{acc}_{\text{nil}}(\bar{e}) \star \text{acc}_{\text{nil}}(\bar{r})]} \text{FRAME}
\end{array}$$

Figure 28. Proof tree for $\bar{r} := \text{CoreApi}_k(c, \bar{e})$.



Definition 11 (Reachability). $\text{reach}_\tau^p(x)$ returns the set of addresses for all heap locations that are transitively reachable from variable x at program point p on trace τ . Hence, $\forall x, \tau, p. \text{val}_\tau(x) \in \text{reach}_\tau^p(x)$ holds for all program points p after x is defined.

$$\forall x, y. pts(x) \cap pts(y) = \emptyset \implies disjoint(\{x, y\})$$

Definition 12 (Allocation site). $as_{\tau}(h)$ returns the allocation site for a heap location h on trace τ , which is the label of the statement that allocated this heap location.

allocated within the APPLICATION by `new()` or allocated within the CORE and returned from a CORE API call.

To distinguish these types of heap locations, we run a pass-through analysis that provides the judgments $pt_{\text{CORE}}^p(a, \tau)$ and $pt_{\text{ret}}^p(a, \tau)$ denoting that a heap location allocated at allocation site a passed through (pt) the return argument of a $c := \text{CoreAlloc}(\bar{e})$ statement and through one of the return arguments $\bar{r}$ of a $\bar{r} := \text{CoreApi_k}(c, \bar{e})$ statement, respectively, between label a and program point p on trace τ . I.e., we have that $pt_{\text{CORE}}^p(as_\tau(val_\tau(c)), \tau)$ and $\forall r \in \text{set}(\bar{r}). pt_{\text{ret}}^p(as_\tau(val_\tau(r)), \tau)$ hold at the program point p on trace τ after executing the statement $c := \text{CoreAlloc}(\bar{e})$ and $\bar{r} := \text{CoreApi_k}(c, \bar{e})$, respectively.

Definition 13 (APPLICATION-managed heap locations). We call a heap location h APPLICATION-managed at program point p on trace τ if h is either allocated within the APPLICATION or has been returned from a $\bar{r} := \text{CoreApi_k}(c, \bar{e})$ statement.

$$am_\tau^p(h) \triangleq is\text{-}app(as_\tau(h)) \vee pt_{\text{ret}}^p(as_\tau(h), \tau)$$

Escape Analysis. The goal of the escape analysis is to correctly place heap locations into `slh`, `*sgh`, and `sih`. In particular, we want to establish globally that an APPLICATION-managed heap location and a CORE instance are in `slh` and `sih`, respectively, if they are *local*.

We first define what it means for a heap location to be local (cf. Def. 15). I.e., this definition takes all threads into account and states that a heap location h is local to a thread t if and only if t is the only thread that can potentially access h .

Locality of a heap location is approximated by our escape analysis. The result of the escape analysis is formalized in a judgement $local^p(x)$ for some variable x and program point p . The intuition is that a variable that is local points to heap locations (i.e., $*x$) that are *only* accessible by the current thread and, thus, can only be modified or even referred to by the current thread. The escape analysis is sound in that no heap location that is accessible by another thread will ever be reported as local (cf. Asm. 5), but potentially imprecise in that some locations that are not accessible by other threads will fail to be local.

Definition 14 (Accessibility). We write $accessible_t^p(h)$ to denote that heap location h is accessible by thread t at program point p . A thread may access such a heap location either directly via variables or indirectly by dereferencing other heap locations. We define accessibility independently of variables and, thus, accessibility of h does not change when variables go out of scope. Instead, accessibility is monotonic for a thread's execution.

Definition 15 (Locality). A heap location h is local at program point p if it is accessible by a single thread t .

$$localhl_t^p(h) \triangleq accessible_t^p(h) \wedge (\forall t'. t' \neq t \implies \neg accessible_{t'}^p(h))$$

Lemma 3 (Uniqueness of locality). The thread t having access to a local heap location h is unique, i.e.,

$$\forall h, t, t', p. localhl_t^p(h) \wedge localhl_{t'}^p(h) \implies t = t'.$$

Proof sketch. The lemma follows directly from Def. 15. $\square$

Lemma 4 (Locality is reverse monotonic). A local heap location h at program point p' must be local at every earlier program point p if h is accessible at p , i.e.,

$$\forall h, t, p, p'. p \preceq p' \wedge accessible_t^p(h) \wedge localhl_t^{p'}(h) \implies localhl_t^p(h).$$

Proof sketch. We prove this lemma by contradiction for arbitrary h, t, p, p' . $\neg localhl_t^p(h)$ implies that h is accessible by another thread t' , i.e., $t' \neq t \wedge accessible_{t'}^p(h)$. Since accessibility is monotonic, h remains accessible by t' at p' contradicting $localhl_t^{p'}(h)$. $\square$

Lemma 5 (Locality is reverse transitive). If a local heap location h' is transitively reachable from another heap location h then h must also be local.

$$\forall h, h', t, \tau, p. localhl_t^p(h') \wedge h' \in reach_\tau^p(h) \implies localhl_t^p(h)$$

Proof sketch. We prove this lemma by contradiction for arbitrary h, h', t, τ, p . I.e., a thread t' exists such that h is accessible by t' . h' is accessible by t' via reachability from h , thus, contradicting $localhl_t^p(h')$. $\square$

Assumption 5 (Soundness of escape analysis). We assume that the escape analysis is sound, i.e., reports a heap location to which variable x points as being local only if the corresponding heap location is indeed local (or x is `nil`) for every possible trace τ , i.e.,

$$\forall x, \tau, p. local^p(x) \implies val_\tau(x) = \text{nil} \vee \exists t. localhl_t^p(val_\tau(x)).$$

Based on these definitions and the soundness of our analyses, we prove several lemmas that relate accessible heap locations to our ghost sets and corollaries that lift these properties to variables and the judgements we obtain from our static analyses. We will later use these corollaries to show that these judgements discharge our proof rules' side conditions ω .

Lemma 6 (Inaccessibility implies set absence). All heap locations stored in the ghost sets are accessible by at least one thread.

$$\forall h, \tau, p. h \neq \text{nil} \wedge p \in \tau \wedge (\forall t. \neg accessible_t^p(h)) \implies \forall t. h \notin (slh_t \cup *sgh \cup sih_t)^p$$

where s^p refers to the set stored in variable s at program point p .

Proof sketch. We prove this lemma by induction over program traces. The base case for the empty trace holds trivially as `slh` and `sih` for every thread t and `*sgh` are initialized to the empty set. In the inductive step, we prove this lemma for an arbitrary heap location h' , program point p' , and trace τ . We assume the premise and apply the induction hypothesis for the immediately preceding program point p

as $\forall t. \neg \text{accessible}_{t'}^{p'}(h')$ implies $\forall t. \neg \text{accessible}_t^p(h')$ due to monotonicity. We show that $\forall t. h' \notin (\text{slh}_t \cup *sgh \cup \text{sih}_t)^{p'}$ holds by analyzing the ghost operations that $\mathbb{A}$ inserts for a statement s . We assume without loss of generality that thread t_s executes $\mathbb{A}(s)$, which transitions from p to p' . We observe that every element that is added to slh_{t_s} , $*sgh$ or sih_t is either the heap location to which a variable accessible by t_s points or a set of heap locations that are reachable from such a variable. Since h' by assumption is not accessible from any thread at p' , $\mathbb{A}$ does not add h' to any ghost set. $\square$

The next lemmas depend on certain requirements for a codebase, which we define next. As we will see, successfully executing the static analyses implies that a codebase meets these requirements.

Lemma 7 (Locality implies set containment for APPLICATION-managed locations). *An APPLICATION-managed heap location h is in thread t 's slh at program point p if h is local, and in $*sgh$ if h is accessible by multiple threads. Both cases hold if a codebase meets the requirements r_τ^p (cf. Fig. 32).*

$$\begin{aligned} \forall h, t, \tau, p. (h \neq \text{nil} \wedge p \in \tau \wedge \text{accessible}_t^p(h) \wedge \\ \text{am}_\tau^p(h) \wedge r_\tau^p) \implies \\ ((\forall t'. t' = t \vee \neg \text{accessible}_{t'}^p(h)) \iff h \in \text{slh}_t^p) \wedge \\ ((\exists t'. t' \neq t \wedge \text{accessible}_{t'}^p(h)) \iff h \in *sgh^p) \end{aligned}$$

Proof sketch. We prove this lemma by induction over program traces. The base case for the empty trace holds trivially as there are no allocated heap locations yet. In the inductive step, we prove this lemma for an arbitrary heap location h' , thread t , program point p' , and trace τ by applying the induction hypothesis to the immediately preceding program point p and showing that we obtain the specified set containment for p' . I.e., we assume the premise and show that

$$\begin{aligned} ((\forall t'. t' = t \vee \neg \text{accessible}_{t'}^{p'}(h')) \iff h' \in \text{slh}_{t'}^{p'}) \wedge \\ ((\exists t'. t' \neq t \wedge \text{accessible}_{t'}^{p'}(h')) \iff h' \in *sgh^{p'}) \end{aligned} \quad (4)$$

holds. We case split on statement s (before applying $\mathbb{A}$) such that executing $\mathbb{A}(s)$ on thread t_s transitions from p to p' . We first note that the restrictions r are monotonic when going backwards on a trace, i.e., r_τ^p follows from $r_{\tau'}^{p'}$.

- $s = \text{skip}$: Since skip does not allocate any heap locations and leaves accessibility unchanged, we get $\text{accessible}_{t'}^{p'}(h')$ and apply the induction hypothesis. Because algorithm $\mathbb{A}$ leaves all ghost sets unmodified, (4) holds.
- $s = x := \text{new}()$: If $h' = \text{val}_\tau(x)$, then $t = t_s$ as $\text{accessible}_{t'}^{p'}(h')$ holds and no other thread can access h' yet. $\mathbb{A}$ adds h' to slh_t and (4) holds as h' is in no other ghost set (by Lemma 6). Otherwise, h' is already allocated at p and we apply the induction hypothesis to obtain (4) as $\mathbb{A}$ neither adds nor removes h' to and from any ghost set.
- $s = x := *e$: Since s neither allocates new heap locations nor changes accessibility of h' , $\text{accessible}_{t'}^{p'}(h')$

holds and we apply the induction hypothesis. If $h' = \text{val}_\tau(e)$, then $\text{accessible}_{t_s}^p(h')$ and, thus, $h' \in (\text{slh}_{t_s} \cup *sgh)^p$ hold. Hence, $\mathbb{A}$ ensures $\forall t'. \text{slh}_{t'}^{p'} = \text{slh}_{t'}^p$ and $\text{slh}^{p'} = \text{slh}^p$. Otherwise, $\mathbb{A}$ neither adds nor removes h' to and from any ghost set.

- $s = *x := e$: Since s does not allocate new heap locations, $\text{am}_\tau^p(h')$ holds. If $\text{val}_\tau(x) = h'$, then h' is accessible by t_s and we apply the induction hypothesis. Since $\mathbb{A}$ leaves h' in the same ghost set, (4) holds. Otherwise, we focus on the case $\text{val}_\tau(x) \in *sgh^p \wedge h' \in \text{reach}_\tau^p(e) \cap \text{slh}_{t_s}^p$ as $\mathbb{A}$ removes in this case h' from slh_{t_s} and for all other cases guarantees that h' remains in the same ghost set. From $h' \in \text{slh}_{t_s}^p$ and our induction hypothesis, we get $t = t_s$ as h' is accessible only by a single thread. Since $\text{accessible}_{t_s}^p(\text{val}_\tau(x))$ and $\text{am}_\tau^p(\text{val}_\tau(x))$ (from r_τ^p) hold, we apply the induction hypothesis and obtain that another thread t' with $t' \neq t_s$ exists that can access $\text{val}_\tau(x)$. However, by writing e to $\text{val}_\tau(x)$, all from e reachable heap locations including h' become accessible from t' at p' . Since h' is accessible at p' from at least two different threads, namely t_s and t' , we have to show that $h' \in *sgh^{p'}$ and that h' is removed from slh_{t_s} , which is guaranteed by $\mathbb{A}$.
- $s = c := \text{CoreAlloc}(\bar{e})$: If $\exists e. e \in \bar{e} \wedge \text{val}_\tau(e) = h'$, we get $\text{local}^{p'}(e)$ from $r_\tau^{p'}$. Thus, $t_s = t$ as only a single thread can access h' . From Asm. 5 and Lemma 4, $\text{localhl}_t^p(h')$ holds and we apply the induction hypothesis to obtain $h' \in \text{slh}_t^p$. $\mathbb{A}$ guarantees that h' remains in slh_t and that h' is not inserted into any other ghost set since $h' \neq \text{val}_\tau(c)$. Otherwise, $\text{accessible}_t^p(h')$ holds because s cannot change h' 's accessibility as the arguments $\bar{e}$ are shallow (cf. Asm. 3) and, thus, s internally does not have access to h' . We apply the induction hypothesis and observe that $\mathbb{A}$ does not change set containment of h' .
- $s = \bar{r} := \text{CoreApi}_k(c, \bar{e})$: We reason similarly as in the case of $\text{CoreAlloc}(\bar{e})$ except that we consider a third case, namely $\exists r. r \in \bar{r} \wedge \text{val}_\tau(r) = h'$. In this case, $r_\tau^{p'}$ guarantees that h' is local and from $\text{accessible}_{t'}^{p'}(h')$ follows that $t = t_s$. h' is a heap location newly allocated by s and $\mathbb{A}$ guarantees that h' is inserted into slh_t . From Lemma 6, we get that h' is in no other ghost set.
- $s = \text{fork}(\bar{x}) \{s'\}$: Let us call the newly spawned thread t'_s with $t'_s \neq t_s$. Since t'_s can access the variables $\bar{x}$, we have $\forall h. h \in \text{reach}_\tau^p(\bar{x}) \implies \text{accessible}_{t_s}^p(h) \wedge \text{accessible}_{t'_s}^{p'}(h)$. If $h' \notin \text{reach}_\tau^p(\bar{x})$, then accessibility of h' does not change by executing s and we apply the induction hypothesis and note that $\mathbb{A}$ does not modify set containment of h' . In particular, h' is not accessible by t'_s and, thus, $h' \notin \text{slh}_{t'_s}^{p'}$ holds as required by (4). Otherwise ($h' \in \text{reach}_\tau^p(\bar{x})$), we have to prove that $\forall t'. h' \notin \text{slh}_{t'}^{p'}$ and $h' \in *sgh^{p'}$ hold. Since $\text{accessible}_{t_s}^p(h')$ holds, we apply the induction hypothesis and case split on whether $h' \in \text{slh}_{t_s}^p$ holds. If so, $\mathbb{A}$ moves h' from slh_{t_s} to $*sgh$, which is sufficient

$$\begin{aligned}
r_\tau^p &\triangleq (\forall s, x, e, \ell. s^\ell = *x := e \wedge \ell \prec p \implies am_\tau^{\text{pre-}\ell}(val_\tau(x))) \wedge \\
&(\forall s, c, e, \bar{e}, \ell. s^\ell = c := \mathbf{CoreAlloc}(\bar{e}) \wedge \ell \prec p \wedge e \in set(\bar{e}) \implies \\
&\quad val_\tau(e) = \mathbf{nil} \vee am_\tau^{\text{pre-}\ell}(val_\tau(e)) \wedge local^{\text{post-}\ell}(e)) \wedge \\
&(\forall s, k, c, e, \bar{e}, r, \bar{r}, \ell. s^\ell = \bar{r} := \mathbf{CoreApi_k}(c, \bar{e}) \wedge \ell \prec p \wedge e \in set(\bar{e}) \wedge r \in set(\bar{r}) \implies \\
&\quad (val_\tau(e) = \mathbf{nil} \vee am_\tau^{\text{pre-}\ell}(val_\tau(e)) \wedge local^{\text{post-}\ell}(e)) \wedge \\
&\quad (val_\tau(c) = \mathbf{nil} \vee \neg am_\tau^{\text{pre-}\ell}(val_\tau(c))) \wedge \\
&\quad (val_\tau(r) = \mathbf{nil} \vee local^{\text{post-}\ell}(r)))
\end{aligned}$$

Figure 32. r_τ^p expresses requirements that all statements in a codebase before program point p on trace τ must satisfy. These requirements allow us to relate properties of heap locations to containment in the ghost sets. In particular, heap write statements must write to APPLICATION-managed heap locations only, arguments that are passed to the CORE (i.e., $\bar{e}$ in $\mathbf{CoreAlloc}(\bar{e})$ and $\bar{r} := \mathbf{CoreApi_k}(c, \bar{e})$ statements) must be APPLICATION-managed and local *after* executing the statement unless they are $\mathbf{nil}$, the CORE instance c must *not* be APPLICATION-managed, and return arguments from the CORE, i.e., $\bar{r}$ in $\bar{r} := \mathbf{CoreApi_k}(c, \bar{e})$, must be local or $\mathbf{nil}$.

as $\forall t'. t' \neq t_s \implies h' \notin \text{slh}_{t'}^p$, holds. Otherwise, $h' \in *sgh^p$ holds and $\mathbb{A}$ ensures $h' \in *sgh^{p'}$. $\square$

Corollary 3 (Set containment in $\text{slh} \cup *sgh$). *A variable x is in a thread t 's slh_t or $*sgh$ at program point p if x is a defined variable, all heap locations x may point to are APPLICATION-managed, and the requirements r_τ^p hold.*

$$\begin{aligned}
&\forall x, t, p, \tau. p \in \tau \wedge \text{defined}_t^p(x) \wedge r_\tau^p \wedge \\
&\quad (\forall h. as_\tau(h) \in \text{pts}(x) \implies am_\tau^p(h)) \implies \\
&\quad val_\tau(x) = \mathbf{nil} \vee val_\tau(x) \in (\text{slh}_t \cup \text{sgh})^p
\end{aligned}$$

where $\text{defined}_t^p(x)$ expresses that x is defined at p for thread t .

Proof sketch. Let x, t, p, τ be arbitrary and assume the corollary's premise. If $val_\tau(x) = \mathbf{nil}$ holds, then the corollary holds trivially. Otherwise, x points at p to an allocated heap location, which we call h' , that is, thus, accessible from thread t i.e., $h' = val_\tau(x) \wedge \text{accessible}_t^p(h')$. From Asm. 4 we obtain $as_\tau(h') \in \text{pts}(x)$ and, thus, $am_\tau^p(h')$ holds. We apply Lemma 7 and observe that one of the equivalences' left-hand sides must be satisfied. Therefore, h' is either in slh_t^p or $*sgh^p$. $\square$

Lemma 8 (Locality implies set containment for CORE instances). *A heap location h at program point p that corresponds to a CORE instance returned from an earlier $\mathbf{CoreAlloc}(\bar{e})$ statement is in a thread t 's sih if h is local and the restrictions r_τ^p (Fig. 32) hold.*

$$\begin{aligned}
&\forall h, t, \tau, p. h \neq \mathbf{nil} \wedge p \in \tau \wedge local^p_t(h) \wedge \\
&\quad pt_{\text{CORE}}^p(h, \tau) \wedge r_\tau^p \implies h \in \text{sih}_t^p
\end{aligned}$$

Proof sketch. We prove this lemma by induction over program traces. The base case for the empty trace holds trivially as there are no allocated heap locations yet. In the inductive step, we prove this lemma for an arbitrary heap location h' , thread t , program point p' , and trace τ by applying the induction hypothesis to the immediately preceding program point p and showing that we obtain the specified set containment for p' . I.e., we assume the premise

and show that $h' \in \text{sih}_t^{p'}$ holds. We case split on statement s (before applying $\mathbb{A}$) such that executing $\mathbb{A}(s)$ on thread t_s transitions from p to p' . We first note that the restrictions r are monotonic when going backwards on a trace, i.e., r_τ^p follows from $r_\tau^{p'}$.

- $s = \mathbf{skip}$: Since $\mathbf{skip}$ does not allocate CORE instances and leaves accessibility unchanged, we get $local^p_t(h')$ and apply the induction hypothesis. We get $h' \in \text{sih}_t^{p'}$ as $\mathbb{A}$ leaves all ghost sets unmodified.
- $s = x := \mathbf{new}()$: $h' \neq val_\tau(x)$ holds because x points to a newly allocated heap location that has not been passed through the return argument of $\mathbf{CoreAlloc}(\bar{e})$. Therefore, $local^p_t(h')$ holds and we apply the induction hypothesis. We observe that $\mathbb{A}$ leaves sih_t unchanged.
- $s = x := *e$: Since s does not allocate CORE instances, $pt_{\text{CORE}}^p(h', t)$ holds and we apply the induction hypothesis. The lemma holds as $\mathbb{A}$ does not modify sih_t .
- $s = *x := e$: Identical reasoning as for reading a heap location.
- $s = c := \mathbf{CoreAlloc}(\bar{e})$: If $val_\tau(c) = h'$, then $local^p_t(h')$ implies $t = t_s$. $\mathbb{A}$ guarantees that $h' \in \text{sih}_t^{p'}$. Otherwise, $local^p_t(h')$ and $pt_{\text{CORE}}^p(h', \tau)$ hold and we apply the induction hypothesis. $h' \in \text{sih}_t^{p'}$ holds as $\mathbb{A}$ does not remove elements from sih .
- $s = \bar{r} := \mathbf{CoreApi_k}(c, \bar{e})$: Since s does not allocate CORE instances, $local^p_t(h')$ and $pt_{\text{CORE}}^p(h', \tau)$ hold and we apply the induction hypothesis. Furthermore, $\mathbb{A}$ does not remove elements from $\text{sih}_{t'}$ for any thread t' .
- $s = \mathbf{fork}(\bar{x}) \{s'\}$: Let us call the newly spawned thread t'_s with $t'_s \neq t_s$. If $accessible_{t'_s}^{p'}(h')$, then $t = t'_s$ as h' is local. However, h' can only be accessible to t'_s if h' is reachable from $\bar{x}$, which is accessible from thread t_s too. I.e., $accessible_{t_s}^{p'}(h')$ holds contradicting $local^p_t(h')$. Otherwise, $\mathbb{A}$ initializing $\text{sih}_{t'_s}$ to the empty set does not violate the lemma as t'_s cannot access h' . Furthermore, we apply the induction hypothesis as $pt_{\text{CORE}}^p(h', \tau)$ holds and we note that $\mathbb{A}$ does not remove any element from sih_t .

□

Corollary 4 (Escape analysis implies set containment in sih). *A variable x is in a thread t 's sih_t at program point p if x is a defined variable, local, all heap locations x may point to passed through the return parameter of some $\text{CoreAlloc}(\bar{e})$, and the requirements r_τ^p hold.*

$$\begin{aligned} \forall x, t, p, \tau. p \in \tau \wedge \text{local}^p(x) \wedge \text{defined}_t^p(x) \wedge r_\tau^p \wedge \\ (\forall h. \text{as}_\tau(h) \in \text{pts}(x) \implies \text{pt}_{\text{CORE}}^p(h, \tau)) \implies \\ \text{val}_\tau(x) = \text{nil} \vee \text{val}_\tau(x) \in \text{sih}_t^p \end{aligned}$$

Proof sketch. Let x, t, p, τ be arbitrary and assume the corollary's premise. If $\text{val}_\tau(x) = \text{nil}$ holds, then the corollary holds trivially. Otherwise, x points at p to an allocated heap location, which we call h' , which is, thus, accessible from thread t , i.e., $h' = \text{val}_\tau(x) \wedge \text{accessible}_t^p(h')$. $\text{localhl}_t^p(h')$ follows from Asm. 5. From Asm. 4 we obtain $\text{as}_\tau(h') \in \text{pts}(x)$ and, thus, $\text{pt}_{\text{CORE}}^p(h', \tau)$. Applying Lemma 8 completes the proof. □

Having defined the properties that successfully executing our static analyses provides, we present next how we apply the static analyses in **DIONON** (Def. 16) and prove in Lemma 10 that this application discharges the side conditions ω (cf. Fig. 13).

As shown in Def. 16, we check for every heap read operation $x := *e$ that e points to APPLICATION-managed heap locations, which are identified by their allocation site a . Analogously, we check for heap writes $*x := e$ that x satisfies the same property. For every $\text{CoreAlloc}(\bar{e})$ and $\bar{r} := \text{CoreApi_k}(c, \bar{e})$, we check that the arguments $\bar{e}$ point to disjoint heap locations and that these heap locations are local and APPLICATION-managed. Additionally, we check for $\bar{r} := \text{CoreApi_k}(c, \bar{e})$ that c points to a local CORE instance, i.e., a local heap location that has been returned by an earlier CORE allocation call, and that the return arguments $\bar{r}$ are local.

Definition 16 (Static analyses for **DIONON**). *In **DIONON**, we execute the static analyses on a codebase to obtain the following judgements for every statement s at label ℓ therein, denoted as $j(s^\ell)$.*

$$\begin{aligned} j(x := *e) &\triangleq \forall a, \tau. a \in \text{pts}(e) \implies \\ &\quad \text{am}_\tau^{\text{pre-}\ell}(a) \\ j(*x := e) &\triangleq \forall a, \tau. a \in \text{pts}(x) \implies \\ &\quad \text{am}_\tau^{\text{pre-}\ell}(a) \end{aligned}$$

$$\begin{aligned} j(c := \text{CoreAlloc}(\bar{e})) &\triangleq \text{disjoint-as}(\bar{e}) \wedge \text{local}_{\text{am}}^\ell(\bar{e}) \\ j(\bar{r} := \text{CoreApi_k}(c, \bar{e})) &\triangleq \text{disjoint-as}(\bar{e}) \wedge \text{local}_{\text{am}}^\ell(\bar{e}) \\ &\quad \wedge \text{local}_{\text{CORE}}^\ell(c) \wedge \text{local}_{\text{ret}}^\ell(\bar{r}) \end{aligned}$$

where

$$\begin{aligned} \text{disjoint-as}(\bar{e}) &\triangleq \forall i, j. 0 \leq i < j < \text{len}(\bar{e}) \implies \\ &\quad \text{pts}(\bar{e}[i]) \cap \text{pts}(\bar{e}[j]) = \emptyset \end{aligned}$$

$$\begin{aligned} \text{local}_{\text{am}}^\ell(\bar{e}) &\triangleq \forall e, h, \tau. e \in \text{set}(\bar{e}) \wedge \\ &\quad \text{as}_\tau(h) \in \text{pts}(e) \implies \\ &\quad \text{local}^{\text{post-}\ell}(e) \wedge \text{am}_\tau^{\text{pre-}\ell}(h) \end{aligned}$$

$$\begin{aligned} \text{local}_{\text{CORE}}^\ell(c) &\triangleq \forall h, \tau. \text{as}_\tau(h) \in \text{pts}(c) \implies \\ &\quad \text{local}^{\text{pre-}\ell}(c) \wedge \text{pt}_{\text{CORE}}^{\text{pre-}\ell}(h, \tau) \\ \text{local}_{\text{ret}}^\ell(\bar{r}) &\triangleq \forall r, \tau. r \in \text{set}(\bar{r}) \implies \text{local}^{\text{post-}\ell}(r) \end{aligned}$$

Lemma 9 (Discharging the requirements). *We show that the judgements provided by our static analyses $j(s^\ell)$ (cf. Def. 16) for every statement s at label ℓ before program point p and our assumptions are sufficient to discharge the requirements r_τ^p (cf. Fig. 32).*

$$\forall p, \tau. p \in \tau \wedge (\forall s, \ell. \ell \prec p \wedge j(s^\ell)) \implies r_\tau^p$$

Proof sketch. We prove this lemma by induction over program traces. The base case for the empty trace holds trivially as there are no preceding statements s^ℓ . In the inductive step, we prove this lemma for an arbitrary program point p' and trace τ by applying the induction hypothesis to the immediately preceding program point p . I.e., we show that $\forall s', \ell'. \ell' \prec p' \wedge j(s'^{\ell'})$ and r_τ^p imply $r_\tau^{p'}$ by case splitting on statement s^ℓ whose execution transitions from p to p' .

- $s^\ell = *x := e$: We have to prove that $\text{am}_\tau^p(\text{val}_\tau(x))$ holds. From $j(s^\ell)$ and Asm. 4, we get $\text{val}_\tau(x) = \text{nil} \vee \text{am}_\tau^p(\text{val}_\tau(x))$. $x \neq \text{nil}$ holds as the statement would otherwise crash (cf. Asm. 2).
- $s^\ell = c := \text{CoreAlloc}(\bar{e})$: We have to show for an arbitrary argument $e \in \text{set}(\bar{e})$ that $\text{val}_\tau(e) = \text{nil} \vee \text{am}_\tau^p(\text{val}_\tau(e)) \wedge \text{local}^{p'}(e)$ holds. If $\text{val}_\tau(e) \neq \text{nil}$, then we apply Asm. 4 to obtain $\text{am}_\tau^p(\text{val}_\tau(e))$ from $\text{local}_{\text{am}}^\ell(\bar{e})$.
- $s^\ell = \bar{r} := \text{CoreApi_k}(c, \bar{e})$: We proceed identically as for $\text{CoreAlloc}(\bar{e})$. Additionally, we have to show $\text{val}_\tau(c) = \text{nil} \vee \neg \text{am}_\tau^p(\text{val}_\tau(c))$ and $\text{val}_\tau(r) = \text{nil} \vee \text{local}^{p'}(r)$ for an arbitrary return argument $r \in \bar{r}$, which we get from $\text{local}_{\text{CORE}}^\ell(c)$ by applying Asm. 4 and $\text{local}_{\text{ret}}^\ell(\bar{r})$.
- otherwise: $r_\tau^{p'}$ holds because no requirements for s^ℓ must be met.

□

Lemma 10 (Discharging the side conditions ω). *We show that the judgements provided by our static analyses $j(s)$ (cf. Def. 16) for every statement s in a codebase c together with our assumptions are sufficient to discharge the side conditions $\omega(s)$ (cf. Fig. 13).*

$$\forall s \in c. (\forall s' \in c. j(s')) \implies \omega(s)$$

Proof sketch. We prove this lemma for an arbitrary statement s at label ℓ such that $s \in c$, assume $\forall s' \in c. j(s')$ and show that $\omega(s)$ holds by case splitting on statement s . Throughout the proof, we use program point p to refer to s 's pre-state, i.e., $p \triangleq \text{pre-}\ell$. We obtain $\forall \tau. r_\tau^p$ from Lemma 9.

- $s = x := *e$: From Cor. 3, we get $\text{val}_\tau(e) = \text{nil} \vee \text{val}_\tau(e) \in (\text{slh} \cup \text{sgh})^p$. e points to an allocated heap location and cannot be nil as the statement would otherwise crash (cf. Asm. 2).

- $s = *x := e$: Analogous to heap reads but for x instead of e .
- $s = c := \text{CoreAlloc}(\bar{e})$: From $\text{disjoint-as}(\bar{e})$, we obtain via Lemma 2 $\text{disjoint}(\bar{e})$. $\text{local}_{\text{am}}^{\ell}(\bar{e})$ allows us to apply Lemma 7 providing $\forall e \in \text{set}(\bar{e}). \text{val}_{\tau}(e) = \text{nil} \vee \text{val}_{\tau}(e) \in \text{slh}^p$. Lastly, $*\text{used} = \text{false}$ holds by our assumption that we have a single CORE allocation statement in the codebase c . We lift this assumption in Sec. B.2.4.
- $s = \bar{r} := \text{CoreApi_k}(c, \bar{e})$: We obtain $\text{disjoint}(\bar{e})$ and $\forall e \in \text{set}(\bar{e}). \text{val}_{\tau}(e) = \text{nil} \vee \text{val}_{\tau}(e) \in \text{slh}^p$ likewise to the previous case for $\text{CoreAlloc}(\bar{e})$. Left to show is $\text{val}_{\tau}(c) = \text{nil} \vee \text{val}_{\tau}(c) \in \text{slh}^p$, which we obtain from $\text{local}_{\text{CORE}}^{\ell}(c)$ by applying Cor. 4.
- otherwise: $\omega(s) = \text{true}$ and, thus, the lemma holds trivially.

□

Corollary 5 (Proof construction). *Successfully executing DIONON's static analyses on a codebase c and the CORE's auto-active verification combined with our assumptions allow us to construct a separation logic proof for c .*

If $\forall s, k. s \in c \wedge j(s) \wedge$
 $(s = c := \text{CoreAlloc}(\bar{e}) \implies$
 $\Pi_g \vdash [P_{\text{CoreAlloc}}(\bar{e})] \ s \ [Q_{\text{CoreAlloc}}(c, \bar{e})]) \wedge$
 $(s = \bar{r} := \text{CoreApi_k}(c, \bar{e}) \implies$
 $\Pi_g \vdash [P_{\text{CoreApi_k}}(c, \bar{e})] \ s \ [Q_{\text{CoreApi_k}}(c, \bar{e}, \bar{r})]),$
 then $\text{emp} \vdash [\phi] \ s_{\text{init}}; \mathbb{A}(c) \ [\text{true}]$

where s_{init} is ghost code creating and initializing the thread-local ghost sets slh and slh for the main thread, as well as the global ghost set $*\text{sgh}$ and the ghost flag $*\text{used}$.

Proof sketch. All our proof rules (cf. Fig. 12) have the same shape, namely $\Pi_g \vdash [\Pi_l] \ \mathbb{A}(s) \ [\Pi_l]$ for a statement s . As shown by Lemma 10, the judgements obtained from the static analyses allow us to discharge the side conditions that are associated with each proof rule (Fig. 13). Therefore, left to show is that we initially establish Π_l and Π_g such that we can compose the proof rules to form a proof for an entire codebase c . The ghost statement s_{init} creates and initializes the ghost sets slh , slh , and $*\text{sgh}$ as well as the ghost flag $*\text{used}$. Thus, we can complete the proof tree as shown in Fig. 33. □

B.2.4. Extensions. Having covered the main soundness result, we discuss two extensions to bridge the gap to realistic applications of DIONON as used in our case studies. We first lift the restriction of at most one CORE instance to allow a codebase to create unboundedly-many CORE instances. Second, we allow the CORE to invoke callbacks into the APPLICATION and discuss the side conditions that arise by this extension.

Unboundedly-many CORE instances. So far, our global program invariant Π_g contains the separating conjunct

$$\text{acc}(\text{used}) \star (\neg(*\text{used}) \implies \phi).$$

As explained in App. B.1, each execution of a protocol role is parameterized by a unique rid . I.e., ϕ and all I/O permissions that ϕ internally provides are parameterized by rid and, thus, are not interchangeable but specific to a particular rid . Hence, we can change the separating conjunct stated above to

$$\text{acc}(\text{used}) \star (\forall \text{rid} \notin *\text{used} \implies \phi(\text{rid}))$$

providing a family of I/O permissions, where used points to a ghost set containing the rids that have already been used. In addition, we adapt the entire program's precondition from ϕ to $\forall \text{rid}. \phi(\text{rid})$ and change the translation $\mathbb{A}(c := \text{CoreAlloc}(\bar{e}))$ to, first, pick a fresh rid' such that $\text{rid}' \notin *\text{used}$ and, second, adding rid' to $*\text{used}$. Picking such a fresh rid' is always possible since rid ranges over $\mathbb{N}$.

Adding callbacks to the CORE. So far, we have treated the statements $\text{CoreAlloc}(\bar{e})$ and $\bar{r} := \text{CoreApi_k}(c, \bar{e})$ as atomic statements in our language. These two statements are internally implemented as sequences of statements, which we hereafter call CORE statements. As these statements constitute the CORE, we auto-actively prove that a particular postcondition holds when control transfers back to the APPLICATION after fully executing these statements.

In the presence of callbacks, however, calling into the CORE becomes non-atomic and control flow might transfer to the APPLICATION before reaching the post-state for which we know that the postcondition holds. We can treat callbacks as temporarily pausing the execution of these auto-actively verified CORE statements to (sequentially) execute some statements belonging to the APPLICATION before eventually resuming execution of CORE statements.

With respect to algorithm $\mathbb{A}$ and the ghost sets, interrupting the execution of CORE statements to execute certain APPLICATION statements s_{app} means that heap locations on which the CORE statements operate are missing from the ghost sets while executing s_{app} as we remove them from the ghost sets before executing CORE statements and put them back only after the CORE statements' postcondition holds. Missing permissions include both arguments $\bar{e}$ and the CORE instance c . Therefore, we have to make sure that s_{app} neither accesses heap locations to which $\bar{e}$ points nor invokes API calls on the CORE instance c as the CORE invariant might not hold.

We can lift these restrictions by introducing additional proof obligations for the auto-active verification. More specifically, if we auto-actively prove that the CORE statements satisfy a particular precondition for the callback, then we can update the ghost sets accordingly. E.g., such a precondition can specify permissions for heap locations passed to the callback or that the CORE invariant holds.

In our SSM Agent case study, we make use of these proof obligations for the callback delivering incoming messages to the APPLICATION as we specify that the CORE transfers permission for the incoming message to the APPLICATION. Conceptually, this allows us to add the corresponding heap location to slh before executing the statements constituting

$$\begin{array}{c}
\vdots \\
\text{emp} \vdash [\text{emp}] \ s_{\text{init}} \ [R_g \star R_l] \\
\hline
\text{emp} \vdash [\phi] \ s_{\text{init}} \ [R_g \star R_l \star \phi] \quad \text{FRAME} \\
\hline
\text{emp} \vdash [\phi] \ s_{\text{init}} \ [\Pi_g \star \Pi_l] \quad \text{CONSEQ} \\
\hline
\text{emp} \vdash [\phi] \ s_{\text{init}}; \mathbb{A}(p) \ [\text{true}]
\end{array}
\quad
\begin{array}{c}
\vdots \text{ Fig. 12} \\
\frac{\Pi_g \vdash [\Pi_l] \ \mathbb{A}(p) \ [\Pi_l]}{\Pi_g \vdash [\Pi_l] \ \mathbb{A}(p) \ [\text{true}]} \text{CONSEQ} \\
\hline
\text{emp} \vdash [\Pi_g \star \Pi_l] \ \mathbb{A}(p) \ [\Pi_g] \quad \text{SHARE} \\
\hline
\text{emp} \vdash [\Pi_g \star \Pi_l] \ \mathbb{A}(p) \ [\text{true}] \quad \text{CONSEQ} \\
\hline
\text{emp} \vdash [\Pi_g \star \Pi_l] \ \mathbb{A}(p) \ [\text{true}] \quad \text{SEQ}
\end{array}$$

$$\text{with } R_l \triangleq \text{slh} = \emptyset \star \text{sih} = \emptyset$$

$$R_g \triangleq \text{acc}(\text{sgh}) \star \text{sg}h = \emptyset \star \text{acc}(\text{used}) \star \text{used} = \text{false}$$

Figure 33. Proof tree showing the initial establishment of Π_l and Π_g for a codebase p . We assume that the ghost statement s_{init} initializes slh and sih to the empty set, as stated in R_l , and allocates two heap locations on the ghost heap storing $\emptyset$ and false to which $\text{sg}h$ and used point, respectively (cf. R_g).

the callback because the auto-active proof guarantees that no statement in the **CORE** thereafter accesses this heap location.

For our case studies, it was not necessary to transfer permissions from a callback back to the **CORE** via a callback's postcondition. Extending **DIODON** to allow such permission

transfers would require an analysis of the callback showing that the **APPLICATION** possesses these permissions while executing callback and that the corresponding heap locations do not get accessed by the **APPLICATION** after the callback returns.